

รายงานโครงงาน

เรื่อง ระบบตรวจวัดแอลกอฮอล์ด้วยเทคโนโลยี IoT

เสนอ

อาจารย์ประเสริฐศักดิ์ อู่อรุญ

จัดทำโดย

นายณันธภัทร์ ลิ่มเจริญ รหัสนิสิต 6621653751 เลขที่ 23

นายณัฐภัทร ยืนยง รหัสนิสิต 6621655231 เลขที่ 35

นายบรรณวัฒน์ จันทร์สมมิตร รหัสนิสิต 6621655274 เลขที่ 38

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 02739496

สาขาวิชาเทคโนโลยีสารสนเทศ คณะศิลปศาสตร์และวิทยาศาสตร์

มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตกำแพงแสน

ภาคเรียนที่ 2 ปีการศึกษา 2568

คำนำ

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา 02739496 จัดทำขึ้นเพื่อนำเสนอโครงงานระบบตรวจวัดแอลกอฮอล์ด้วยเทคโนโลยี IoT ซึ่งใช้บอร์ด ESP32 ร่วมกับเซนเซอร์ MQ-3 ตรวจวัดไอแอลกอฮอล์ในลมหายใจ แสดงผลบนจอ OLED และไฟ LED ส่งข้อมูลผ่านแพลตฟอร์ม NETPIE ด้วยโปรโตคอล MQTT ที่เข้ารหัสด้วย TLS และจัดเก็บประวัติการตรวจบน Cloud Firestore โดยมีแอปพลิเคชันที่พัฒนาด้วย Flutter สำหรับเจ้าหน้าที่ใช้ตรวจผู้ขับขี่ทีละขั้นตอน บันทึกผล ดูสรุปผล และตั้งค่าเครื่องวัดจากระยะไกล

เนื้อหาของรายงานแบ่งออกเป็น 5 บท ได้แก่ บทนำ ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง การออกแบบและพัฒนาระบบ ผลการดำเนินงาน และสรุปผลและข้อเสนอแนะ โดยอธิบายการทำงานของระบบตามซอร์สโค้ดเวอร์ชันล่าสุดที่เผยแพร่บน GitHub พร้อมแผนภาพสถาปัตยกรรม ผังงาน ภาพหน้าจอแอปพลิเคชัน และผลการทดสอบ รวมทั้งแนบซอร์สโค้ดเฟิร์มแวร์ไว้ในภาคผนวก

คณะผู้จัดทำขอขอบพระคุณอาจารย์ประเสริฐศักดิ์ อู่อรุญ ที่ให้ความรู้และคำแนะนำตลอดการจัดทำโครงงาน หวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์แก่ผู้ที่สนใจการประยุกต์ใช้เทคโนโลยี IoT ในงานด้านความปลอดภัย หากมีข้อผิดพลาดประการใด คณะผู้จัดทำขออภัยมา ณ ที่นี้

คณะผู้จัดทำ

สารบัญ

หน้า

คำนำ ..... ก

สารบัญ ..... ข

สารบัญตาราง ..... ง

สารบัญภาพ ..... จ

บทที่ 1 บทนำ 1

1.1 ที่มาและความสำคัญ 1

1.2 วัตถุประสงค์ 1

1.3 ขอบเขตของโครงงาน 1

1.4 ประโยชน์ที่คาดว่าจะได้รับ 2

1.5 นิยามศัพท์เฉพาะ 2

บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง 4

2.1 อินเทอร์เน็ตของสรรพสิ่ง (Internet of Things) 4

2.2 ปริมาณแอลกอฮอล์ในเลือดและเกณฑ์ตามกฎหมาย 4

2.3 เซนเซอร์ตรวจวัดแอลกอฮอล์ MQ-3 4

2.4 บอร์ดไมโครคอนโทรลเลอร์ ESP32 4

2.5 จอแสดงผล OLED และการสื่อสารแบบ I2C 5

2.6 โปรโตคอล MQTT และแพลตฟอร์ม NETPIE 5

2.7 Firebase 5

2.8 Flutter และการจัดการสถานะด้วย Provider 6

2.9 การเข้ารหัสการสื่อสารด้วย TLS 6

2.10 พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล 6

บทที่ 3 การออกแบบและพัฒนาระบบ 7

3.1 สถาปัตยกรรมของระบบ 7

3.2 อุปกรณ์ที่ใช้ในการพัฒนา 8

3.3 การต่อวงจร 8

3.4 เครื่องมือและไลบรารีที่ใช้ในการพัฒนา 9
3.5 การพัฒนาเฟิร์มแวร์ 10
3.6 การสื่อสารข้อมูลผ่าน MQTT 14
3.7 การออกแบบการจัดเก็บข้อมูล 15
3.8 การพัฒนาแอปพลิเคชัน 16
3.9 ขั้นตอนการพัฒนา 19
บทที่ 4 ผลการดำเนินงาน 20
4.1 ต้นแบบอุปกรณ์ 20
4.2 แอปพลิเคชัน 20
4.3 การตั้งค่า Cloud Firestore 22
4.4 ผลการทดสอบ 23
4.5 ภาพการดำเนินงาน 24
บทที่ 5 สรุปผล อภิปรายผล และข้อเสนอแนะ 25
5.1 สรุปผล 25
5.2 อภิปรายผล 25
5.3 ปัญหาและข้อจำกัด 25
5.4 ข้อเสนอแนะ 26
บรรณานุกรม 27
ภาคผนวก ก ที่เก็บซอร์สโค้ดและวิธีติดตั้ง 29
ภาคผนวก ข ซอร์สโค้ดเฟิร์มแวร์ 30

สารบัญตาราง

ตาราง **หน้า**

1.1 นิยามศัพท์เฉพาะ 2
2.1 เกณฑ์ปริมาณแอลกอฮอล์ในเลือดของผู้ขับขี่ 4
2.2 คุณสมบัติของ ESP32 ที่เกี่ยวข้องกับโครงงาน 4
3.1 อุปกรณ์ที่ใช้ในการพัฒนา 8
3.2 การเชื่อมต่ออุปกรณ์กับขาของบอร์ด ESP32 9
3.3 เครื่องมือและไลบรารีที่ใช้ในการพัฒนา 9
3.4 ระดับการแจ้งเตือน 12
3.5 ค่าคงที่สำคัญในเฟิร์มแวร์ 14
3.6 หัวข้อ MQTT ที่ใช้ในระบบ 14
3.7 โครงสร้างเอกสารในคอลเลกชัน measurements 15
3.8 โครงสร้างซอร์สโค้ดของแอปพลิเคชัน 16
3.9 สถานะการเชื่อมต่อเครื่องวัดในแอปพลิเคชัน 17
3.10 เงื่อนไขการตรวจสอบข้อมูลผู้ถูกตรวจ 18
4.1 ผลการทดสอบการทำงานของระบบ 23
4.2 ผลการตรวจสอบการคอมไพล์และคุณภาพของซอร์สโค้ด 24

สารบัญภาพ

ภาพ **หน้า**

3.1 สถาปัตยกรรมของระบบตรวจวัดแอลกอฮอล์ 7
3.2 ผังงานการทำงานของเฟิร์มแวร์บน ESP32 11
4.1 ต้นแบบอุปกรณ์ตรวจวัดแอลกอฮอล์ 20
4.2 หน้าเข้าสู่ระบบและหน้าหลักของแอปพลิเคชัน 21
4.3 การตรวจผู้ขับขี่ 3 ขั้นในแอปพลิเคชัน 21
4.4 หน้าประวัติ รายละเอียดรายการ และหน้าเครื่องวัด 22
4.5 Security Rules ของ Cloud Firestore บน Firebase Console 22
4.6 ภาพการดำเนินงานของคณะผู้จัดทำ 24

บทที่ 1
บทนำ

1.1 ที่มาและความสำคัญ

ปัญหาการดื่มเครื่องดื่มแอลกอฮอล์แล้วขับขี่ยังคงเป็นสาเหตุสำคัญของอุบัติเหตุบนท้องถนน กฎหมายไทยจึงกำหนดเกณฑ์ปริมาณแอลกอฮอล์ในเลือดของผู้ขับขี่ไว้อย่างชัดเจน โดยผู้ขับขี่ทั่วไปต้องมีปริมาณแอลกอฮอล์ในเลือดไม่เกิน 50 มิลลิกรัมเปอร์เซ็นต์ และผู้ขับขี่บางกลุ่ม เช่น ผู้ที่อายุต่ำกว่า 20 ปี หรือผู้ที่มีใบอนุญาตขับรถชั่วคราว ต้องไม่เกิน 20 มิลลิกรัมเปอร์เซ็นต์ (กฎกระทรวง ฉบับที่ 21, 2560) การตรวจวัดเบื้องต้นที่ทำได้รวดเร็วจึงช่วยคัดกรองและป้องกันความเสี่ยงก่อนการขับขี่

คณะผู้จัดทำจึงพัฒนาระบบตรวจวัดแอลกอฮอล์ในลมหายใจด้วยเทคโนโลยีอินเทอร์เน็ตของสรรพสิ่ง (Internet of Things: IoT) ประกอบด้วยอุปกรณ์ที่ใช้บอร์ด ESP32 ร่วมกับเซนเซอร์ MQ-3 ซึ่งแสดงผลทันทีบนจอ OLED และไฟ LED ส่งข้อมูลผ่านแพลตฟอร์ม NETPIE ด้วยโปรโตคอล MQTT และแอปพลิเคชันที่พัฒนาด้วย Flutter สำหรับเจ้าหน้าที่ใช้ตรวจผู้ขับขี่ตามขั้นตอน บันทึกข้อมูลผู้เข้ารับการตรวจ ดูสรุปผล และปรับเกณฑ์การแจ้งเตือนจากระยะไกล โดยจัดเก็บประวัติบน Cloud Firestore ที่ต้องเข้าสู่ระบบก่อนจึงจะเข้าถึงได้ ระบบนี้สามารถประยุกต์ใช้ในสถานศึกษา องค์กร หรือจุดคัดกรองเบื้องต้น

1.2 วัตถุประสงค์

1. เพื่อพัฒนาระบบตรวจวัดระดับแอลกอฮอล์ในลมหายใจโดยใช้เทคโนโลยี IoT
2. เพื่อแสดงผลค่าที่ตรวจวัดได้แบบเรียลไทม์ทั้งบนจอแสดงผลของอุปกรณ์และบนแอปพลิเคชัน
3. เพื่อจัดเก็บข้อมูลการตรวจวัดลงในฐานข้อมูลบน Firebase อย่างปลอดภัย
4. เพื่อศึกษาและประยุกต์ใช้บอร์ดไมโครคอนโทรลเลอร์ ESP32 ร่วมกับแพลตฟอร์ม IoT
5. เพื่อเพิ่มแนวทางในการป้องกันอุบัติเหตุที่เกิดจากการดื่มแอลกอฮอล์แล้วขับขี่

1.3 ขอบเขตของโครงงาน

1. ด้านฮาร์ดแวร์ ใช้บอร์ด ESP32 DOIT DevKit V1 เซนเซอร์ MQ-3 จอ OLED ขนาด 128×64 จุด แบบ I2C และไฟ LED 2 ดวงสำหรับแจ้งเตือน
2. ด้านเฟิร์มแวร์ พัฒนาด้วยภาษา C++ บน Arduino Framework ผ่าน PlatformIO ทำหน้าที่อ่านค่าเซนเซอร์ ประมวลผล แสดงผล ส่งข้อมูลขึ้นคลาวด์ และรับค่า Threshold จากแอปพลิเคชัน
3. ด้านการสื่อสาร ใช้แพลตฟอร์ม NETPIE เป็นตัวกลางรับส่งข้อมูลผ่าน MQTT ที่เข้ารหัสด้วย TLS ทั้งฝั่งอุปกรณ์และฝั่งแอปพลิเคชัน
4. ด้านแอปพลิเคชัน พัฒนาด้วย Flutter ภาษา Dart รองรับ Android, iOS, macOS และเว็บ ประกอบด้วยหน้าเข้าสู่ระบบ และ 3 แท็บหลัก ได้แก่ หน้าหลัก หน้าประวัติ และหน้าเครื่องวัด โดยการตรวจแต่ละครั้งทำเป็น 3 ขั้น คือ กรอกข้อมูล เป่า และดูผลตรวจ
5. ด้านการจัดเก็บข้อมูล ใช้ Cloud Firestore เก็บประวัติผู้เข้ารับการตรวจ ใช้ Firebase Authentication ยืนยันตัวตนเจ้าหน้าที่ และใช้ Security Rules จำกัดสิทธิ์การเข้าถึง ส่วน

Firebase Realtime Database เป็นตัวเลือกเสริมของเฟิร์มแวร์ซึ่งปิดไว้เป็นค่าเริ่มต้น

6. ระบบเป็นต้นแบบเพื่อการศึกษา ค่าที่วัดได้เป็นค่าดัชนีในสเกล 0–100 ซึ่งยังไม่ได้สอบเทียบกับเครื่องวัดมาตรฐาน จึงไม่สามารถใช้แทนเครื่องตรวจวัดที่รับรองตามกฎหมาย

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ช่วยลดความเสี่ยงในการเกิดอุบัติเหตุบนท้องถนนจากการเมาแล้วขับ
2. เจ้าหน้าที่ตรวจและบันทึกผลได้รวดเร็ว ลดความผิดพลาดจากการจดบันทึกด้วยมือ
3. มีฐานข้อมูลประวัติการตรวจที่จำกัดสิทธิ์การเข้าถึง และส่งออกเป็นไฟล์ CSV เพื่อนำไปวิเคราะห์ต่อได้
4. คณะผู้จัดทำได้พัฒนาทักษะการเขียนโปรแกรมควบคุม ESP32 การพัฒนาแอปพลิเคชันด้วย Flutter การออกแบบความปลอดภัยของระบบ และการประยุกต์ใช้เทคโนโลยี IoT

1.5 นิยามศัพท์เฉพาะ

ตารางที่ 1.1 นิยามศัพท์เฉพาะ

คำศัพท์	ความหมาย
IoT (Internet of Things)	การเชื่อมต่ออุปกรณ์ทางกายภาพเข้ากับเครือข่ายอินเทอร์เน็ต เพื่อรับส่งข้อมูลและควบคุมการทำงานได้
ไมโครคอนโทรลเลอร์	คอมพิวเตอร์ขนาดเล็กบนชิปเดียวที่ใช้ควบคุมอุปกรณ์อิเล็กทรอนิกส์ ในโครงงานนี้คือ ESP32
ADC (Analog-to-Digital Converter)	วงจรแปลงสัญญาณแอนะล็อก เช่น แรงดันจากเซนเซอร์ ให้เป็นตัวเลขดิจิทัล
MQTT	โปรโตคอลรับส่งข้อความแบบ Publish/Subscribe ที่มีขนาดเล็ก เหมาะกับอุปกรณ์ IoT
NETPIE	แพลตฟอร์ม IoT ที่ให้บริการ MQTT Broker, Device Shadow และ REST API
Device Shadow	ข้อมูลสถานะล่าสุดของอุปกรณ์ที่แพลตฟอร์มเก็บไว้ ให้ระบบอื่นอ่านได้แม้อุปกรณ์ไม่ได้เชื่อมต่ออยู่
Heartbeat	ข้อความสถานะที่อุปกรณ์ส่งเป็นรอบ ๆ แม้ไม่มีการเป่า เพื่อให้แอปพลิเคชันรู้ว่าอุปกรณ์ยังทำงานอยู่
TLS (Transport Layer Security)	โปรโตคอลเข้ารหัสการสื่อสารบนเครือข่าย และยืนยันตัวตนของเซิร์ฟเวอร์ด้วยใบรับรองดิจิทัล (Certificate)
Security Rules	กฎของ Cloud Firestore ที่กำหนดว่าใครอ่านหรือเขียนข้อมูลใดได้ และข้อมูลต้องมีรูปแบบอย่างไร
Offline-first	แนวทางออกแบบที่ให้แอปพลิเคชันบันทึกข้อมูลในเครื่องได้ก่อน แล้วซิงก์ขึ้นคลาวด์เมื่อเชื่อมต่อได้

คำศัพท์	ความหมาย
Threshold	ค่าเกณฑ์ที่ใช้แบ่งระดับการแจ้งเตือน ในระบบนี้มี 2 ค่า
BAC (Blood Alcohol Concentration)	ปริมาณแอลกอฮอล์ในเลือด ในระบบนี้ใช้เรียกค่าดัชนี 0–100 ที่แปลงจากเซนเซอร์ ซึ่งไม่ใช่ค่า BAC ตามกฎหมาย

บทที่ 2
ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

2.1 อินเทอร์เน็ตของสรรพสิ่ง (Internet of Things)

IoT คือโครงสร้างพื้นฐานที่เชื่อมต่อสิ่งต่าง ๆ ทั้งทางกายภาพและทางดิจิทัลเข้าด้วยกันผ่านเทคโนโลยีสารสนเทศและการสื่อสาร (International Telecommunication Union [ITU], 2012) ระบบ IoT โดยทั่วไปประกอบด้วย 3 ส่วน ได้แก่ อุปกรณ์ที่ตรวจวัดหรือควบคุมสิ่งแวดล้อม เครือข่ายและแพลตฟอร์มที่รับส่งและจัดเก็บข้อมูล และแอปพลิเคชันที่นำข้อมูลไปแสดงผลหรือใช้ตัดสินใจ ซึ่งโครงงานนี้ออกแบบตามโครงสร้างดังกล่าว

2.2 ปริมาณแอลกอฮอล์ในเลือดและเกณฑ์ตามกฎหมาย

กฎกระทรวง ฉบับที่ 21 (พ.ศ. 2560) ออกตามความในพระราชบัญญัติจราจรทางบก พ.ศ. 2522 กำหนดเกณฑ์ที่ถือว่าผู้ขับขี่เมาสุรา ดังตารางที่ 2.1 (กฎกระทรวง ฉบับที่ 21, 2560; มติชนออนไลน์, 2560)

ตารางที่ 2.1 เกณฑ์ปริมาณแอลกอฮอล์ในเลือดของผู้ขับขี่

กลุ่มผู้ขับขี่	ถือว่าเมาสุราเมื่อเกิน
ผู้ขับขี่ที่อายุต่ำกว่า 20 ปีบริบูรณ์ ผู้ที่มีใบอนุญาตขับรถชั่วคราว ผู้ที่ไม่มีใบอนุญาตขับขี่ หรือถูกพักใช้หรือเพิกถอนใบอนุญาต	20 มิลลิกรัมเปอร์เซ็นต์
ผู้ขับขี่ทั่วไป	50 มิลลิกรัมเปอร์เซ็นต์

เกณฑ์ดังกล่าวเป็นเหตุผลที่ระบบในโครงงานนี้ออกแบบให้มีค่า Threshold 2 ระดับ โดยมีค่าเริ่มต้นเป็น 20 และ 50 และให้ผู้ใช้ปรับค่าได้จากแอปพลิเคชัน

2.3 เซนเซอร์ตรวจวัดแอลกอฮอล์ MQ-3

MQ-3 เป็นเซนเซอร์ก๊าซชนิดสารกึ่งตัวนำที่ใช้ดีบุกออกไซด์ (SnO_2) เป็นวัสดุตรวจวัด เมื่อมีไอแอลกอฮอล์ ค่าความต้านทานของเซนเซอร์จะลดลงตามความเข้มข้น โมดูลจึงให้แรงดันเอาต์พุตแบบแอนะล็อกที่สูงขึ้นเมื่อแอลกอฮอล์มากขึ้น เซนเซอร์ตรวจวัดแอลกอฮอล์ได้ในช่วงประมาณ 0.05–10 มิลลิกรัมต่อลิตร ใช้ขดลวดให้ความร้อนที่แรงดัน 5 โวลต์ ต้องอุ่นเซนเซอร์ (Preheat) ก่อนใช้งาน และค่าที่วัดได้ขึ้นกับอุณหภูมิและความชื้นของสภาพแวดล้อม (Hanwei Electronics, n.d.)

2.4 บอร์ดไมโครคอนโทรลเลอร์ ESP32

ESP32 เป็นไมโครคอนโทรลเลอร์ของบริษัท Espressif Systems ที่มีวงจร Wi-Fi และ Bluetooth ในตัว จึงเหมาะกับงาน IoT คุณสมบัติที่เกี่ยวข้องกับโครงงานแสดงดังตารางที่ 2.2

ตารางที่ 2.2 คุณสมบัติของ ESP32 ที่เกี่ยวข้องกับโครงงาน

คุณสมบัติ	รายละเอียด
หน่วยประมวลผล	Xtensa LX6 แบบ 32 บิต 2 แกน ความเร็วสูงสุด 240 MHz

คุณสมบัติ	รายละเอียด
หน่วยความจำ	SRAM 520 KB และหน่วยความจำแฟลชสำหรับเก็บโปรแกรมและข้อมูลถาวร (NVS)
การเชื่อมต่อไร้สาย	Wi-Fi 802.11 b/g/n และ Bluetooth
วงจรแปลงแอนะล็อกเป็นดิจิทัล	ADC แบบ SAR ความละเอียด 12 บิต (ค่า 0–4095) จำนวน 2 ชุด คือ ADC1 และ ADC2
การสื่อสารกับอุปกรณ์ต่อพ่วง	รองรับ I2C, SPI และ UART

ที่มา: Espressif Systems (n.d.-a)

ในการต่อเซนเซอร์ มีข้อจำกัดสำคัญ 2 ประการ คือ (1) ADC2 ถูกใช้งานโดยวงจร Wi-Fi ทำให้อ่านค่าไม่ได้ขณะเชื่อมต่อ Wi-Fi และ (2) ช่วงแรงดันที่อ่านได้ขึ้นกับค่าการลดทอนสัญญาณ (Attenuation) ที่ตั้งไว้ (Espressif Systems, n.d.-b) โครงงานนี้จึงเลือกขา GPIO 34 ซึ่งเป็นช่องของ ADC1 และตั้งค่าการลดทอนเป็น 11 dB เพื่อให้อ่านแรงดันได้กว้างที่สุด นอกจากนี้ ESP32 มีพื้นที่เก็บข้อมูลแบบ Non-volatile Storage (NVS) ในหน่วยความจำแฟลช ซึ่งข้อมูลไม่หายเมื่อรีบูต (Espressif Systems, n.d.-c) โครงงานใช้พื้นที่นี้เก็บค่า Threshold ล่าสุด

2.5 จอแสดงผล OLED และการสื่อสารแบบ I2C

จอ OLED ขนาด 128×64 จุดที่ใช้ควบคุมด้วยชิป SSD1306 ซึ่งสื่อสารกับไมโครคอนโทรลเลอร์ผ่านบัส I2C โดยใช้สายสัญญาณเพียง 2 เส้น คือ SDA (ข้อมูล) และ SCL (สัญญาณนาฬิกา) และอ้างอิงอุปกรณ์ด้วยหมายเลขที่อยู่ (Address) ซึ่งโมดูลที่ใช้มีที่อยู่ 0x3C (Solomon Systech, 2008)

2.6 โปรโตคอล MQTT และแพลตฟอร์ม NETPIE

MQTT เป็นโปรโตคอลรับส่งข้อความแบบ Publish/Subscribe ผ่านตัวกลางที่เรียกว่า Broker ผู้ส่ง (Publisher) ส่งข้อความไปยังหัวข้อ (Topic) และผู้รับ (Subscriber) ที่สมัครรับหัวข้อนั้นจะได้รับข้อความทันที โดยไม่ต้องรู้จักกันโดยตรง MQTT กำหนดระดับการรับประกันการส่ง (Quality of Service: QoS) ไว้ 3 ระดับ คือ 0, 1 และ 2 และรองรับข้อความแบบ Retained ที่ Broker เก็บไว้ส่งให้ผู้รับรายใหม่ (Banks & Gupta, 2014)

NETPIE เป็นแพลตฟอร์ม IoT ที่ให้บริการ MQTT Broker โดยกำหนดให้หัวข้อสำหรับรับส่งข้อความระหว่างอุปกรณ์ขึ้นต้นด้วย @msg/ ซึ่งอุปกรณ์ที่ส่งถึงกันได้ต้องอยู่ในกลุ่ม (Group) เดียวกัน และมีบริการ Device Shadow สำหรับเก็บสถานะล่าสุดของอุปกรณ์ ซึ่งอุปกรณ์อัปเดตผ่านหัวข้อ @shadow/data/update NETPIE รองรับการเชื่อมต่อ MQTT แบบเข้ารหัส TLS ที่พอร์ต 8883 และผ่าน WebSocket แบบเข้ารหัสที่พอร์ต 443 สำหรับแอปพลิเคชันบนเว็บ (NETPIE, n.d.)

2.7 Firebase

Firebase เป็นแพลตฟอร์มบริการฝั่งเซิร์ฟเวอร์ของ Google โครงงานนี้ใช้บริการหลัก 2 ส่วน ได้แก่ Cloud Firestore ซึ่งเก็บข้อมูลเป็นเอกสาร (Document) ภายในคอลเลกชัน (Collection) และ Firebase Authentication สำหรับยืนยันตัวตนผู้ใช้ด้วยอีเมลและรหัสผ่าน เมื่อเข้าสู่ระบบสำเร็จ ผู้ใช้จะได้รับ ID Token

ซึ่งแนบไปกับคำขอ REST API ของ Firestore เพื่อให้ Security Rules ตรวจสอบสิทธิ์ได้ (Google, n.d.-a, n.d.-d, n.d.-e) นอกจากนี้ Firebase ยังมี Realtime Database ที่เก็บข้อมูลเป็นโครงสร้าง JSON แบบต้นไม้ (Google, n.d.-b) ซึ่งเฟิร์มแวร์รองรับเป็นตัวเลือกเสริม

2.8 Flutter และการจัดการสถานะด้วย Provider

Flutter เป็นชุดพัฒนาซอฟต์แวร์ของ Google สำหรับสร้างแอปพลิเคชันหลายแพลตฟอร์ม เช่น Android, iOS, macOS และเว็บ จากซอร์สโค้ดชุดเดียวด้วยภาษา Dart (Google, n.d.-c) ส่วน Provider เป็นแพ็กเกจจัดการสถานะ (State Management) ที่ให้หน้าจอหลายหน้าใช้ข้อมูลชุดเดียวกัน และอัปเดตหน้าจอโดยอัตโนมัติเมื่อข้อมูลเปลี่ยน (Rousselet, n.d.)

2.9 การเข้ารหัสการสื่อสารด้วย TLS

TLS เป็นโปรโตคอลที่ทำให้การสื่อสารระหว่างไคลเอนต์และเซิร์ฟเวอร์ถูกเข้ารหัส ป้องกันการดักอ่านและแก้ไขข้อมูลระหว่างทาง และให้ไคลเอนต์ยืนยันได้ว่ากำลังติดต่อกับเซิร์ฟเวอร์ตัวจริง โดยตรวจใบรับรองของเซิร์ฟเวอร์ย้อนกลับไปจนถึงใบรับรองหลัก (Root CA) ที่ไคลเอนต์เชื่อถือ (Rescorla, 2018) อุปกรณ์ขนาดเล็กอย่าง ESP32 ไม่มีชุดใบรับรองหลักแบบระบบปฏิบัติการทั่วไป จึงต้องฝังใบรับรองหลักของเซิร์ฟเวอร์ที่ใช้ไว้ในเฟิร์มแวร์ และต้องรู้เวลาปัจจุบันเพื่อตรวจว่าใบรับรองยังไม่หมดอายุ

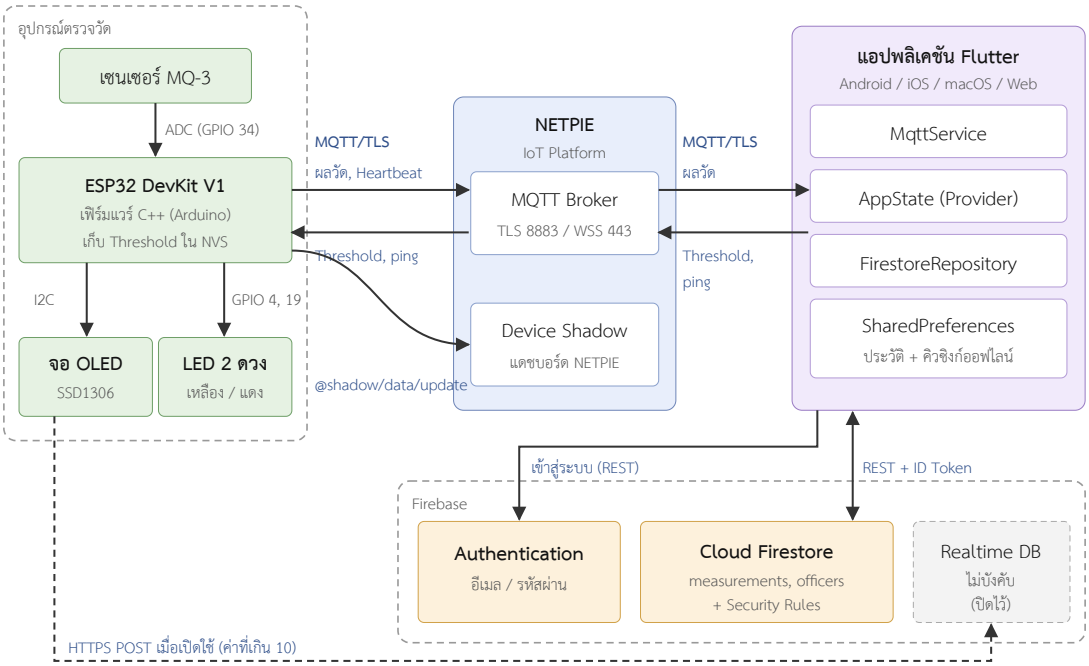
2.10 พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล

พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562 กำหนดให้ผู้เก็บรวบรวมข้อมูลส่วนบุคคล เช่น ชื่อ-นามสกุล และเลขประจำตัวประชาชน ต้องมีมาตรการรักษาความมั่นคงปลอดภัยที่เหมาะสม จำกัดการเข้าถึงเฉพาะผู้ที่มีหน้าที่ และเก็บข้อมูลเท่าที่จำเป็น (พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล, 2562) ระบบนี้เก็บข้อมูลผู้เข้ารับการตรวจ จึงออกแบบให้ต้องเข้าสู่ระบบก่อนเข้าถึงข้อมูล จำกัดสิทธิ์ด้วย Security Rules และปกปิดเลขบัตรประชาชนบางส่วนเมื่อแสดงผลและส่งออกข้อมูล

บทที่ 3
การออกแบบและพัฒนาระบบ

3.1 สถาปัตยกรรมของระบบ

ระบบประกอบด้วย 4 ส่วนหลัก ได้แก่ อุปกรณ์ตรวจวัด แพลตฟอร์ม NETPIE แอปพลิเคชัน Flutter และบริการ Firebase ซึ่งเชื่อมโยงกันดังภาพที่ 3.1 (Catboyz-45, 2026a, 2026b)



ภาพที่ 3.1 สถาปัตยกรรมของระบบตรวจวัดแอลกอฮอล์
ที่มา: คณะผู้จัดทำ

ลำดับการไหลของข้อมูลในระบบเป็นดังนี้

1. ESP32 อ่านค่าจากเซนเซอร์ MQ-3 ทุก 1 วินาที ประมวลผลเป็นค่าดัชนี 0–100 แสดงผลบนจอ OLED และควบคุมไฟ LED ตามระดับแจ้งเตือน แม้ไม่ได้เชื่อมต่ออินเทอร์เน็ต
2. เมื่อค่าที่วัดได้เกิน 10 ESP32 ส่งผลวัดไปที่หัวข้อ @msg/home/device_status และเมื่อไม่มีการเป่าจะส่งข้อความเดียวกันเป็น Heartbeat ทุก 5 วินาที นอกจากนี้ยังอัปเดต Device Shadow ทุกครั้งที่อ่านค่า เพื่อใช้ดูบนแดชบอร์ดของ NETPIE
3. แอปพลิเคชันเชื่อมต่อ NETPIE ด้วยอุปกรณ์ (Device) ของตัวเองที่อยู่กลุ่มเดียวกับบอร์ด รับข้อความผ่าน MQTT แบบ TLS แล้วส่งเข้าสถานะกลาง ทำให้ทุกหน้าจออัปเดตตรงกัน และใช้เวลาที่ได้รับข้อความล่าสุดตัดสินว่าเครื่องวัดออนไลน์หรือเงียบไป
4. เจ้าหน้าที่เข้าสู่ระบบด้วย Firebase Authentication แล้วตรวจผู้ขับขี่ตาม 3 ขั้นตอน เมื่อบันทึก แอปเก็บข้อมูลในเครื่องก่อนแล้วส่งขึ้น Cloud Firestore หากส่งไม่สำเร็จจะเข้าคิวรอส่งใหม่อัตโนมัติ

5. เจ้าหน้าที่ปรับค่า Threshold ในแอปพลิเคชัน แอปส่งค่าผ่าน MQTT ไปที่หัวข้อ @msg/home/device_control บอร์ดบันทึกค่าลงหน่วยความจำถาวรแล้วตอบสถานะกลับทันที แอปจึงยืนยันได้ว่าบอร์ดรับค่าใหม่แล้ว

3.2 อุปกรณ์ที่ใช้ในการพัฒนา

อุปกรณ์ฮาร์ดแวร์ที่ใช้ในการพัฒนาต้นแบบแสดงดังตารางที่ 3.1

ตารางที่ 3.1 อุปกรณ์ที่ใช้ในการพัฒนา

ลำดับ	อุปกรณ์	ภาพอุปกรณ์	จำนวน
1	บอร์ด ESP32 DOIT DevKit V1 ไมโครคอนโทรลเลอร์พร้อม Wi-Fi ใช้ประมวลผลหลัก		1 บอร์ด
2	เซนเซอร์ MQ-3 ตรวจวัดไอแอลกอฮอล์ในลมหายใจ		1 ตัว
3	จอแสดงผล OLED 128×64 แบบ I2C ชิปควบคุม SSD1306 ใช้แสดงค่าที่วัดได้		1 จอ
4	หลอดไฟ LED สีเหลืองและสีแดง ใช้แจ้งเตือนระดับ 1 และ 2		2 ดวง
5	ตัวต้านทาน 220 โอห์ม จำกัดกระแสของหลอดไฟ LED ดวงละ 1 ตัว		2 ตัว
6	Breadboard ใช้ต่อวงจรทดลอง		1 แผ่น
7	สายจัมเปอร์ แบบผู้–เมีย ใช้เชื่อมต่ออุปกรณ์		2 ชุด

3.3 การต่อวงจร

การต่ออุปกรณ์เข้ากับขาของบอร์ด ESP32 เป็นไปตามค่าคงที่ที่กำหนดไว้ในเฟิร์มแวร์ ดังตารางที่ 3.2

ตารางที่ 3.2 การเชื่อมต่ออุปกรณ์กับขาของบอร์ด ESP32

อุปกรณ์	ขา ESP32	หน้าที่
MQ-3 (Analog Output)	GPIO 34	รับสัญญาณแอนะล็อกเข้า ADC1
LED สีเหลือง (ผ่านตัวต้านทาน 220 โอห์ม)	GPIO 4	แจ้งเตือนระดับ 1
LED สีแดง (ผ่านตัวต้านทาน 220 โอห์ม)	GPIO 19	แจ้งเตือนระดับ 2
OLED SDA	GPIO 21	สายข้อมูล I2C
OLED SCL	GPIO 22	สายสัญญาณนาฬิกา I2C

ที่มา: Catboyz-45 (2026b)

ข้อควรระวังในการต่อวงจร คือ ขา ADC ของ ESP32 รับแรงดันได้ไม่เกินแรงดันใช้งาน 3.3 โวลต์ หากจ่ายไฟให้โมดูล MQ-3 ที่ 5 โวลต์ แรงดันเอาต์พุตอาจสูงเกินช่วงดังกล่าว จึงควรใช้วงจรแบ่งแรงดัน (Voltage Divider) ก่อนต่อเข้าขา GPIO 34 (Espressif Systems, n.d.-a)

3.4 เครื่องมือและไลบรารีที่ใช้ในการพัฒนา

ฝั่งเฟิร์มแวร์พัฒนาบน PlatformIO ร่วมกับ Arduino Framework (PlatformIO Labs, n.d.) โดยใช้ไลบรารี PubSubClient สำหรับการเชื่อมต่อ MQTT (O'Leary, n.d.) และ ArduinoJson สำหรับสร้างและแยกข้อมูล JSON (Blanchon, n.d.) ส่วนฝั่งแอปพลิเคชันพัฒนาด้วย Flutter SDK และตรวจคุณภาพโค้ดอัตโนมัติด้วย GitHub Actions เครื่องมือและเวอร์ชันทั้งหมดแสดงดังตารางที่ 3.3

ตารางที่ 3.3 เครื่องมือและไลบรารีที่ใช้ในการพัฒนา

ส่วนงาน	เครื่องมือ / ไลบรารี	เวอร์ชัน	หน้าที่
ทั่วไป	Visual Studio Code, Git, GitHub Actions	–	แก้ไขโค้ด จัดการเวอร์ชัน และตรวจโค้ดอัตโนมัติทุก Pull Request
เฟิร์มแวร์	PlatformIO, Arduino Framework (espressif32)	–	คอมไพล์และอัปโหลดโปรแกรมลงบอร์ด
เฟิร์มแวร์	PubSubClient	^2.8	เชื่อมต่อ MQTT
เฟิร์มแวร์	ArduinoJson	^6.21.3	สร้างและแยกข้อมูล JSON
เฟิร์มแวร์	Adafruit SSD1306, Adafruit GFX	^2.5.10, ^1.11.10	ควบคุมจอ OLED
เฟิร์มแวร์	WiFiClientSecure, Preferences (มากับ ESP32 Arduino Core)	–	เชื่อมต่อแบบ TLS และเก็บค่าใน NVS
แอปพลิเคชัน	Flutter SDK (Dart)	3.47.7 (Dart ^3.7.2)	พัฒนาแอปพลิเคชันหลายแพลตฟอร์ม

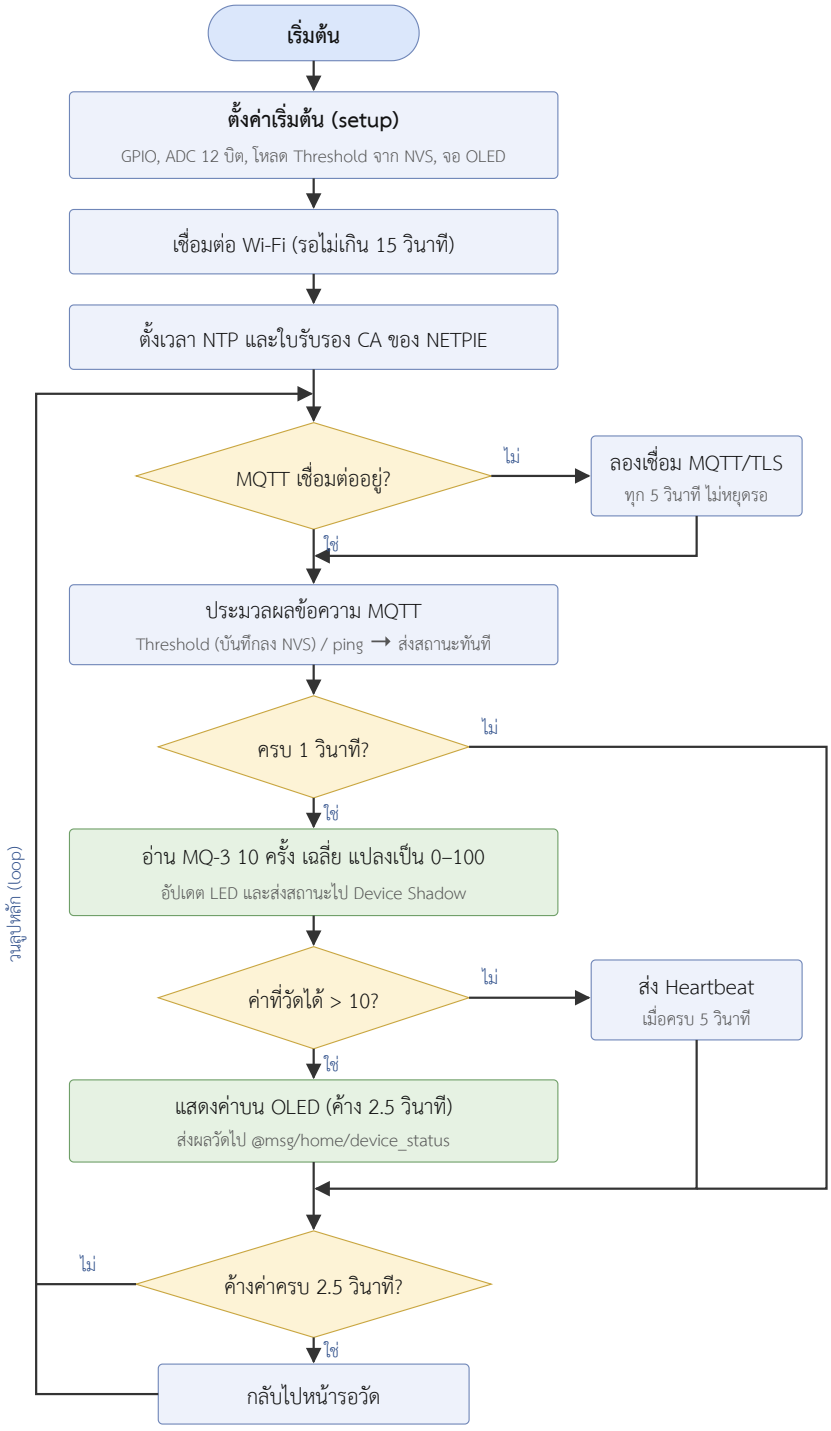
ส่วนงาน	เครื่องมือ / ไลบรารี	เวอร์ชัน	หน้าที่
แอปพลิเคชัน	provider	^6.1.5	จัดการสถานะกลาง
แอปพลิเคชัน	mqtt_client	^10.8.0	รับส่งข้อมูลผ่าน MQTT (TLS / WebSocket)
แอปพลิเคชัน	http	^1.5.0	เรียก REST API ของ Firebase Authentication และ Firestore
แอปพลิเคชัน	shared_preferences	^2.5.3	เก็บประวัติและคิวซิงก์ไว้ในเครื่อง
แอปพลิเคชัน	share_plus, web	^13.3.1, ^1.1.2	แชร์และดาวน์โหลดไฟล์ CSV
แอปพลิเคชัน	IBM Plex Sans Thai (SIL OFL 1.1)	–	ฟอนต์ของแอปพลิเคชัน ฝังไว้ในแอปเพื่อใช้ได้แม้ออฟไลน์
บริการคลาวด์	NETPIE, Firebase (Authentication, Cloud Firestore)	–	รับส่งข้อมูล ยืนยันตัวตน และจัดเก็บข้อมูล

ที่มา: ไฟล์ platformio.ini และ pubspec.yaml ของโครงงาน (Catboyz-45, 2026a, 2026b)

3.5 การพัฒนาเฟิร์มแวร์

3.5.1 ลำดับการทำงานของเฟิร์มแวร์

เฟิร์มแวร์ทำงานแบบไม่หยุดรอ (Non-blocking) โดยใช้ฟังก์ชัน millis() จับเวลาแทนการหน่วงเวลา ทั้งการอ่านเซนเซอร์ การส่ง Heartbeat และการเชื่อมต่อ MQTT ใหม่ ทำให้บอร์ดยังอ่านค่าและควบคุมไฟเตือนได้ตามปกติแม้อินเทอร์เน็ตขาด ลำดับการทำงานแสดงดังภาพที่ 3.2



ภาพที่ 3.2 ผังงานการทำงานของเฟิร์มแวร์บน ESP32
ที่มา: คณะผู้จัดทำ

3.5.2 การอ่านค่าและแปลงค่าจากเซนเซอร์

ในแต่ละรอบ เฟิร์มแวร์อ่านค่าจากขา GPIO 34 จำนวน 10 ครั้ง ห่างกันครั้งละ 5 มิลลิวินาที แล้วหาค่าเฉลี่ยเพื่อลดสัญญาณรบกวน ได้ค่าดิบ (Raw) ในช่วง 0–4095 ตามความละเอียด 12 บิตของ ADC จากนั้น

แปลงเป็นค่าดัชนีแอลกอฮอล์ในสเกล 0–100 แบบเชิงเส้น จำกัดค่าไม่ให้เกินช่วง และปัดเป็นทศนิยม 1 ตำแหน่ง ตามสูตร

ค่าดัชนีแอลกอฮอล์ = (ค่าดิบ ÷ 4095) × 100

ตัวอย่างโค้ดที่ 3.1 ฟังก์ชันอ่านค่าและแปลงค่าจากเซนเซอร์

```
// อ่านค่าเซ็นเซอร์หลายครั้งแล้วเฉลี่ย เพื่อลดสัญญาณรบกวน
int readAlcoholRaw() {
  long total = 0;
  const int sampleCount = 10;
  for (int sample = 0; sample < sampleCount; sample++) {
    total += analogRead(MQ3_PIN);
    delay(5);
  }
  return total / sampleCount;
}

// แปลงค่าดิบเป็นเปอร์เซ็นต์ ตามสเกล 0–100
float convertRawToBac(int rawValue) {
  return clampBac((rawValue / 4095.0f) * BAC_MAX_VALUE);
}
```

3.5.3 การแบ่งระดับการแจ้งเตือน

ระบบเปรียบเทียบค่าที่วัดได้กับ Threshold 2 ค่า ค่าเริ่มต้นคือ 20 และ 50 เพื่อแบ่งระดับการแจ้งเตือนเป็น 3 ระดับ ดังตารางที่ 3.4 เมื่อได้รับ Threshold ใหม่ เฟิร์มแวร์รับเฉพาะค่าที่เป็นตัวเลข จำกัดค่าให้อยู่ในช่วง 0–100 และสลับค่าให้ Threshold 1 ไม่มากกว่า Threshold 2 เสมอ แอปพลิเคชันใช้กติกาเดียวกันในการแสดงสถานะ

ตารางที่ 3.4 ระดับการแจ้งเตือน

ระดับ	เงื่อนไข	LED เหลือง	LED แดง	สถานะในแอป
0	ค่า < Threshold 1	ดับ	ดับ	SAFE (ปลอดภัย)
1	Threshold 1 ≤ ค่า < Threshold 2	ติด	ดับ	WARNING (เฝ้าระวัง)
2	ค่า ≥ Threshold 2	ติด	ติด	DRUNK (เกินเกณฑ์)

ตัวอย่างโค้ดที่ 3.2 ฟังก์ชันอัปเดตระดับแจ้งเตือนและควบคุม LED

```
// อัปเดตระดับแจ้งเตือน และควบคุม LED ตามค่า BAC
void updateAlertOutputs(float bacValue) {
  if (bacValue >= threshold2) {
    alertLevel = 2;
  } else if (bacValue >= threshold1) {
    alertLevel = 1;
  } else {
    alertLevel = 0;
  }
  digitalWrite(LED1_PIN, alertLevel >= 1 ? HIGH : LOW);
  digitalWrite(LED2_PIN, alertLevel >= 2 ? HIGH : LOW);
}
```

3.5.4 การแสดงผลบนจอ OLED

จอ OLED แสดงข้อมูล 4 บรรทัด ได้แก่ ชื่อระบบ ค่าที่วัดได้ด้วยตัวอักษรขนาดใหญ่ ค่า Threshold ทั้งสอง และสถานะการทำงาน เมื่อวัดได้ค่าเกิน 10 จอจะค้างค่าที่วัดได้ไว้ 2.5 วินาที พร้อมข้อความ "Sent to cloud" จากนั้นกลับไปหน้ารอวัดที่แสดงข้อความ "Waiting for sample" เนื่องจากจอค้างค่าเพียงช่วงสั้น แอปพลิเคชันจึงจำค่าสูงสุดของการเป่าแต่ละครั้งไว้ให้เจ้าหน้าที่บันทึกได้ทัน

3.5.5 การเก็บค่า Threshold ในหน่วยความจำถาวร

เมื่อได้รับ Threshold ใหม่ เฟิร์มแวร์บันทึกค่าลงพื้นที่ NVS ผ่านไลบรารี Preferences และโหลดกลับมาใช้ตอนเริ่มทำงาน ค่าที่เจ้าหน้าที่ตั้งไว้จึงไม่ถูกรีเซ็ตกลับเป็น 20 และ 50 ทุกครั้งที่บอร์ดรีบูตหรือไฟดับ หลังบันทึกแล้วเฟิร์มแวร์ส่งสถานะกลับไปที่ @msg/home/device_status ทันที เพื่อให้แอปพลิเคชันยืนยันได้ว่าบอร์ดใช้ค่าใหม่แล้ว

3.5.6 การเชื่อมต่อเครือข่ายและความปลอดภัย

ตอนเริ่มทำงาน บอร์ดรอเชื่อมต่อ Wi-Fi ไม่เกิน 15 วินาที หากยังเชื่อมต่อไม่ได้จะทำงานต่อแบบออฟไลน์และให้ ESP32 เชื่อมต่อใหม่เอง ส่วน MQTT จะลองเชื่อมต่อใหม่ทุก 5 วินาทีโดยไม่หยุดรอใน loop การเชื่อมต่อ MQTT ใช้ WiFiClientSecure ผ่าน TLS ที่พอร์ต 8883 และตรวจใบรับรองของ NETPIE กับใบรับรองหลักที่ฝังไว้ในไฟล์ ca_certs.h พร้อมตั้งเวลาจากเซิร์ฟเวอร์ NTP เพื่อให้ตรวจวันหมดอายุของใบรับรองได้ หากเชื่อมต่อไม่สำเร็จ เฟิร์มแวร์จะพิมพ์สาเหตุจาก TLS ออกทาง Serial Monitor เพื่อช่วยแก้ปัญหา

ตัวอย่างโค้ดที่ 3.3 การตั้งค่าการเชื่อมต่อ MQTT แบบ TLS ในฟังก์ชัน setup()

```
// ตั้งเวลาจาก NTP ไว้ให้ตรวจวันหมดอายุ certificate ได้ (ทำงานเบื้องหลัง ไม่บล็อก)
configTime(0, 0, "pool.ntp.org", "time.google.com");

mqttSecureClient.setCACert(NETPIE_ROOT_CA);
mqttSecureClient.setHandshakeTimeout(TLS_HANDSHAKE_TIMEOUT_S);
mqttClient.setServer(mqttServer, mqttPort);   // mqtt.netpie.io:8883
mqttClient.setCallback(callback);
mqttClient.setBufferSize(512);
```

3.5.7 ค่าคงที่สำคัญในเฟิร์มแวร์

ตารางที่ 3.5 ค่าคงที่สำคัญในเฟิร์มแวร์

ค่าคงที่	ค่า	ความหมาย
SENSOR_READ_INTERVAL_MS	1,000 ms	ช่วงเวลาระหว่างการอ่านเซนเซอร์แต่ละรอบ
DISPLAY_HOLD_MS	2,500 ms	ระยะเวลาค้างค่าที่วัดได้บนจอ
HEARTBEAT_INTERVAL_MS	5,000 ms	ส่งสถานะให้แอปอย่างน้อยทุก 5 วินาที
MQTT_RETRY_INTERVAL_MS	5,000 ms	ช่วงเวลาลองเชื่อมต่อ MQTT ใหม่
WIFI_CONNECT_TIMEOUT_MS	15,000 ms	เวลารอเชื่อมต่อ Wi-Fi ตอนเริ่มทำงาน
TLS_HANDSHAKE_TIMEOUT_S	10 s	เวลารอการเจรจา TLS สูงสุด
mqttPort	8883	พอร์ต MQTT แบบ TLS ของ NETPIE
MIN_REPORT_BAC	10	ค่าขั้นต่ำที่ถือว่ามีการเป่าและส่งผลวัด
BAC_MAX_VALUE	100	ค่าสูงสุดของสเกล
threshold1, threshold2	20, 50	ค่าเริ่มต้นของ Threshold (ถูกแทนด้วยค่าที่บันทึกใน NVS)
MQTT Buffer Size	512 bytes	ขนาดบัฟเฟอร์ข้อความ MQTT

ที่มา: Catboyz-45 (2026b)

3.6 การสื่อสารข้อมูลผ่าน MQTT

อุปกรณ์และแอปพลิเคชันเชื่อมต่อ Broker ของ NETPIE (mqtt.netpie.io) เป็นอุปกรณ์คนละตัวที่อยู่ในกลุ่มเดียวกัน แอปจึงไม่ต้องใช้รหัสของบอร์ด หัวข้อที่ใช้แสดงดังตารางที่ 3.6

ตารางที่ 3.6 หัวข้อ MQTT ที่ใช้ในระบบ

ทิศทาง	หัวข้อ (Topic)	หน้าที่
แอป → บอร์ด	@msg/home/device_control	ส่งค่า Threshold หรือ {"ping": 1} เพื่อขอสถานะทันที
บอร์ด → แอป	@msg/home/device_status	ส่งผลวัดเมื่อมีการเป่า Heartbeat ทุก 5 วินาที และตอบกลับ ping หรือการตั้งค่า Threshold
บอร์ด → NETPIE	@shadow/data/update	อัปเดต Device Shadow ทุก 1 วินาที สำหรับแดชบอร์ดของ NETPIE (แอปไม่ได้อ่าน)

ที่มา: Catboyz-45 (2026a, 2026b)

ตัวอย่างโค้ดที่ 3.4 ข้อมูลสถานะที่อุปกรณ์ส่ง (JSON)

```
{
  "bac": 32.5,
  "raw": 1331,
  "threshold1": 20,
  "threshold2": 50,
  "alertLevel": 1,
  "millis": 120000
}
```

ข้อมูลข้างต้นประกอบด้วยค่าดัชนีแอลกอฮอล์ ค่าดิบจาก ADC ค่า Threshold ทั้งสอง ระดับการแจ้งเตือน และเวลาที่ผ่านไปนับจากบอร์ดเริ่มทำงาน แอปพลิเคชันสมัครรับหัวข้อนี้แบบ QoS 0 และส่งคำสั่งแบบ QoS 1 คำสั่งปรับ Threshold ใช้ JSON ในรูปแบบ {"threshold1": 20, "threshold2": 50} ซึ่งเฟิร์มแวร์รองรับข้อมูลที่ซ้อนอยู่ในฟิลด์ data ด้วย แอปบนมือถือและ macOS เชื่อมต่อที่พอร์ต 8883 แบบ TLS ส่วนแอปบนเว็บเชื่อมต่อผ่าน WebSocket แบบเข้ารหัสที่ wss://mqtt.netpie.io:443/mqtt

3.7 การออกแบบการจัดเก็บข้อมูล

3.7.1 Cloud Firestore

แอปพลิเคชันบันทึกประวัติผู้เข้ารับการตรวจในคอลเลกชัน measurements ผ่าน REST API โดยแนบ ID Token ของเจ้าหน้าที่ที่เข้าสู่ระบบ ใช้เวลาที่บันทึกในหน่วยมิลลิวินาทีเป็นรหัสเอกสาร และอ่านข้อมูลทีละหน้า (Pagination) โครงสร้างเอกสารแสดงดังตารางที่ 3.7

ตารางที่ 3.7 โครงสร้างเอกสารในคอลเลกชัน measurements

ฟิลด์	ชนิดข้อมูล	ความหมายและเงื่อนไข
id	string	รหัสรายการ ต้องตรงกับรหัสเอกสาร
name	string	ชื่อ-นามสกุลผู้เข้ารับการตรวจ (1–120 ตัวอักษร)
idCard	string	เลขบัตรประชาชน ตัวเลข 13 หลัก
age	integer	อายุ 1–120 ปี
licensePlate	string	ทะเบียนรถ (1–20 ตัวอักษร)
province	string	จังหวัดของทะเบียนรถ
alcoholValue	double	ค่าดัชนีแอลกอฮอล์ 0–100
status	string	SAFE, WARNING หรือ DRUNK
timestamp	timestamp	วันและเวลาที่ตรวจ
createdBy	string	รหัสผู้ใช้ (UID) ของเจ้าหน้าที่ที่บันทึก ต้องตรงกับผู้ที่เข้าสู่ระบบ

ที่มา: Catboyz-45 (2026a)

3.7.2 การควบคุมสิทธิ์ด้วย Security Rules

การเข้าสู่ระบบอย่างเดียวยังไม่พอ เพราะผู้ที่มี API Key ของเว็บสามารถสมัครบัญชีเองได้ ระบบจึงกำหนดให้ผู้ที่อ่าน เขียน หรือลบข้อมูลในคอลเลกชัน measurements ได้ ต้องมีเอกสาร officers/{uid} ของตนเอง ซึ่งผู้ดูแลเพิ่มจาก Firebase Console เท่านั้น การเขียนข้อมูลต้องมีฟิลด์ครบตามตารางที่ 3.7 ชนิดข้อมูลและช่วงค่าถูกต้อง และ createdBy ต้องเป็นผู้ที่เข้าสู่ระบบอยู่ ส่วนคอลเลกชันอื่นถูกปิดทั้งหมด กฎทั้งหมดอยู่ในไฟล์ firestore.rules และมีชุดทดสอบที่รันกับ Firestore Emulator

3.7.3 การจัดเก็บข้อมูลในเครื่องและการซิงก์แบบออฟไลน์

แอปพลิเคชันออกแบบแบบ Offline-first เมื่อบันทึกหรือลบรายการ แอปจะปรับข้อมูลในเครื่องด้วย SharedPreferences ก่อน แล้วพยายามส่งขึ้น Firestore หากส่งไม่สำเร็จ รายการจะถูกทำเครื่องหมายว่ารอซิงก์ และส่งใหม่อัตโนมัติเมื่อเชื่อมต่อได้ การลบที่ยังไม่สำเร็จจะเก็บไว้ในคิวแยก เพื่อไม่ให้รายการที่ลบแล้วกลับมาปรากฏอีก สำหรับรายการที่ซิงก์ขึ้นคลาวด์แล้ว แอปจะเก็บเลขบัตรประชาชนในเครื่องเพียง 3 หลักท้าย และล้างข้อมูลในเครื่องทั้งหมดเมื่อออกจากระบบ

3.7.4 Firebase Realtime Database (ตัวเลือกเสริม)

เฟิร์มแวร์ยังรองรับการส่งผลวัดด้วยคำสั่ง HTTPS POST ไปยังโหนด /measurements ของ Realtime Database โดยตรวจใบรับรองของ Google ด้วย แต่ปิดไว้เป็นค่าเริ่มต้น เนื่องจากแอปพลิเคชันไม่ได้อ่านข้อมูลส่วนนี้ การเปิดใช้ต้องตั้งกฎให้เขียนได้โดยไม่ต้องเข้าสู่ระบบ และการส่ง HTTPS แบบรอผลทำให้ loop ช้าลง ผู้ที่ต้องการใช้สามารถเปิดได้ด้วย build flag ENABLE_FIREBASE_RTDB=1

3.8 การพัฒนาแอปพลิเคชัน

3.8.1 โครงสร้างซอร์สโค้ด

ตารางที่ 3.8 โครงสร้างซอร์สโค้ดของแอปพลิเคชัน

ไฟล์	หน้าที่
main.dart	จุดเริ่มต้นของแอป ตรวจสถานะการเข้าสู่ระบบ และสร้างสถานะกลางของแต่ละผู้ใช้
config.dart	รวมค่าคอนฟิกที่ส่งผ่าน --dart-define
device_reading.dart	โมเดลและตัวแปลงข้อมูลจากบอร์ด (ข้อตกลงข้อมูลกับเฟิร์มแวร์)
device_link.dart, mqtt_service.dart	เชื่อมต่อ NETPIE ผ่าน MQTT พร้อมเชื่อมต่อใหม่อัตโนมัติ ส่ง Threshold และ ping และอุปกรณ์จำลองสำหรับโหมดสาธิต
mqtt_client_factory*.dart	เลือกการเชื่อมต่อตามแพลตฟอร์ม (TLS สำหรับแอป และ WebSocket สำหรับเว็บ)
auth_service.dart	เข้าสู่ระบบด้วย Firebase Authentication และต่ออายุ Token
firestore_repository.dart	อ่าน เขียน และลบข้อมูลใน Firestore ผ่าน REST API
app_state.dart	สถานะกลาง ประวัติ คิวซิงก์ออฟไลน์ สถานะเครื่องวัด และค่าสูงสุดของการเป่า

ไฟล์	หน้าที่
thai_id.dart	ตรวจหลักตรวจสอบและปกปิดเลขบัตรประชาชน
measurement_summary.dart, csv_export*.dart	คำนวณสรุปผลตามช่วงวัน สร้างไฟล์ CSV และดาวน์โหลดหรือแชร์ไฟล์
theme.dart, widgets/common.dart	ชุดสี ฟอนต์ และส่วนประกอบหน้าจอที่ใช้ร่วมกัน
login_page.dart	หน้าเข้าสู่ระบบและทางเข้าโหมดสาธิต
home_page.dart, dashboard_page.dart	แถบเมนู 3 แท็บ และหน้าหลัก
measure_flow_page.dart	การตรวจ 3 ขั้น (ข้อมูล → เป่า → ผลตรวจ) และมาตรวัด
measurement_history_page.dart	หน้าประวัติ สรุปผล ค้นหา กรอง และส่งออก CSV
settings_page.dart	หน้าเครื่องวัด สถานะการเชื่อมต่อ การตั้ง Threshold และบัญชีผู้ใช้

ที่มา: Catboyz-45 (2026a)

3.8.2 การรับข้อมูลจากอุปกรณ์

แอปพลิเคชันสมัครรับหัวข้อ @msg/home/device_status และส่ง {"ping": 1} ทุกครั้งที่เชื่อมต่อสำเร็จ เพื่อให้บอร์ดตอบสถานะล่าสุดทันทีโดยไม่ต้องรอ Heartbeat ข้อมูลที่ได้รับจะถูกส่งเข้าสถานะกลางชุดเดียว แอปแบ่งสถานะการเชื่อมต่อออกเป็น 4 แบบ ดังตารางที่ 3.9 ระหว่างการตรวจ แอปจำค่าสูงสุดที่เกิน 10 ของการเป่าครั้งนั้นไว้ เพราะบอร์ดค้างค่าบนจอเพียง 2.5 วินาที

ตารางที่ 3.9 สถานะการเชื่อมต่อเครื่องวัดในแอปพลิเคชัน

สถานะ	เงื่อนไข
กำลังเชื่อมต่อ	ยังไม่เคยได้รับข้อมูลจากบอร์ด
ออนไลน์	ได้รับข้อมูลจากบอร์ดภายใน 12 วินาทีล่าสุด (เผื่อ Heartbeat หายได้ 1 รอบ)
ไม่ส่งข้อมูล	เคยได้รับข้อมูล แต่บอร์ดเงียบไปเกิน 12 วินาที ค่าที่แสดงอาจไม่ใช่ค่าปัจจุบัน
ไม่ได้เชื่อมต่อ	แอปเชื่อมต่อ MQTT ไม่ได้

ที่มา: Catboyz-45 (2026a)

3.8.3 หน้าจอของแอปพลิเคชัน

แอปพลิเคชันออกแบบให้เจ้าหน้าที่ใช้งานที่จุดตรวจได้รวดเร็ว ประกอบด้วยหน้าจอดังนี้

1. หน้าเข้าสู่ระบบ ใช้อีเมลและรหัสผ่านของเจ้าหน้าที่ และมีทางเข้าโหมดสาธิตที่ใช้เครื่องวัดจำลอง
2. หน้าหลัก แสดงสถานะเครื่องวัด ปุ่ม "เริ่มตรวจผู้ขับขี่" สรุปผลของวันนี้ (จำนวนที่ตรวจ จำนวนที่เกินเกณฑ์ และค่าเฉลี่ย) และรายการตรวจล่าสุด

3. การตรวจ 3 ขั้น ขั้นที่ 1 กรอกข้อมูลผู้ถูกตรวจ ขั้นที่ 2 ให้เป่าโดยมีมาตรวัดแบบวงกลมแบ่งโซนสีตาม Threshold และขั้นที่ 3 แสดงผลตรวจด้วยสีสถานะเต็มครึ่งจอพร้อมตัวเลขขนาดใหญ่ ค่าที่ใช้จะถูกล็อกเมื่อกด "ใช้ค่านี้ ดูผล" จึงไม่เปลี่ยนตามค่าที่บอร์ดยังส่งเข้ามา เจ้าหน้าที่ให้เป่าก่อนแล้วค่อยกรอกข้อมูลได้ แต่แอปจะไม่ยอมบันทึกจนกว่าจะกรอกข้อมูลครบและถูกต้องตามตารางที่ 3.10
4. หน้าประวัติ เลือกช่วงเวลา วันนี้ 7 วัน 30 วัน หรือทั้งหมด แสดงสรุปจำนวนที่ตรวจ สัดส่วนที่เกินเกณฑ์ ค่าเฉลี่ย และค่าสูงสุด ค้นหาด้วยชื่อหรือทะเบียน กรองตามสถานะ แบ่งรายการตามวัน และส่งออกรายการที่แสดงเป็นไฟล์ CSV ที่เปิดใน Excel ได้ โดยเลขบัตรประชาชนถูกปกปิดเสมอ
5. หน้าเครื่องวัด แสดงค่าปัจจุบันและสถานะการเชื่อมต่อ ปรับ Threshold ด้วยแถบเลื่อนที่มี 2 จุดหรือพิมพ์ตัวเลข และปุ่มออกจากระบบ

ตารางที่ 3.10 เงื่อนไขการตรวจสอบข้อมูลผู้ถูกตรวจ

ข้อมูล	เงื่อนไข
ชื่อ-นามสกุล	ต้องไม่เป็นค่าว่าง
เลขบัตรประชาชน	ตัวเลข 13 หลัก และหลักสุดท้ายต้องตรงกับหลักตรวจสอบ (Checksum) ที่คำนวณจาก 12 หลักแรก
อายุ	ตัวเลขจำนวนเต็มระหว่าง 1–120 ปี
ทะเบียนรถ	ต้องไม่เป็นค่าว่าง
จังหวัด	เลือกจากรายการ 77 จังหวัด ซึ่งพิมพ์ค้นหาได้

ที่มา: Catboyz-45 (2026a)

3.8.4 การส่งค่า Threshold ไปยังอุปกรณ์

เมื่อเจ้าหน้าที่ปรับ Threshold แล้วกดส่ง แอปจะจัดลำดับให้ค่าน้อยเป็น Threshold 1 แล้วส่งข้อความ JSON ผ่าน MQTT ไปที่หัวข้อ @msg/home/device_control บอร์ดบันทึกค่าลง NVS และตอบสถานะกลับทันที เมื่อค่าในสถานะตรงกับค่าที่ส่ง แอปจะแสดงว่าบอร์ดใช้ค่าใหม่แล้ว หากไม่ได้รับการยืนยันภายใน 15 วินาที แอปจะแจ้งว่าบอร์ดยังไม่ตอบกลับ

3.8.5 การจัดการข้อมูลสำหรับเชื่อมต่อ

ข้อมูลสำหรับเชื่อมต่อ เช่น รหัส Wi-Fi รหัสอุปกรณ์ NETPIE และค่า Firebase ถูกแยกออกจากซอร์สโค้ด ฝั่งเฟิร์มแวร์เก็บไว้ในไฟล์ secrets.h และฝั่งแอปพลิเคชันส่งผ่านไฟล์ config.json ด้วยคำสั่ง --dart-define-from-file โดยไฟล์ทั้งสองถูกยกเว้นไม่ให้อัปโหลดขึ้นที่เก็บโค้ด แอปใช้อุปกรณ์ NETPIE ของตัวเองแยกจากบอร์ด จึงไม่ต้องฝังรหัสของบอร์ดไว้ในแอป ส่วนเว็บเดโมที่เปิดสาธารณะ build โดยไม่ใส่รหัส NETPIE จึงใช้เครื่องวัดจำลองเสมอ และไม่มีผู้ใดสั่งงานบอร์ดจริงผ่านเว็บได้

3.8.6 การออกแบบส่วนติดต่อผู้ใช้

ส่วนติดต่อผู้ใช้ออกแบบใหม่ทั้งหมดโดยยึดการใช้งานกลางแจ้ง ใช้ธีมสว่างที่มีความต่างของสีสูง และมีธีมมืด สีทั้งหมดกำหนดเป็นชุดสีกลางที่เดียว สถานะทั้ง 3 ระดับใช้สีเขียว เหลืองอำพัน และแดงพร้อมไอคอน

และข้อความกำกับ เพื่อไม่ให้ต้องพึ่งการแยกสีเพียงอย่างเดียว ใช้ฟอนต์ IBM Plex Sans Thai ที่ฝังไว้ในแอป และจัดหน้าจอให้ปุ่มสำคัญอยู่ด้านล่างในระยะนิ้วโป้ง

3.8.7 โหมดสาธิต

หากไม่ได้ตั้งค่าอุปกรณ์ หรือผู้ใช้เลือก "ทดลองใช้โหมดสาธิต" แอปจะใช้เครื่องวัดจำลองที่มีปุ่ม "จำลองการเป่า" และเก็บข้อมูลไว้ในเครื่องแยกจากข้อมูลจริง ทำให้ทดลองใช้งานได้ครบทุกขั้นตอนโดยไม่ต้องมีฮาร์ดแวร์ โหมดนี้ใช้ทั้งในการนำเสนอผลงานและในเว็บเดโมบน GitHub Pages

3.9 ขั้นตอนการพัฒนา

1. ต่อวงจรบน Breadboard โดยต่อเซนเซอร์ MQ-3 เข้ากับขา Analog ของ ESP32 ต่อจอ OLED ผ่าน I2C และต่อหลอดไฟ LED ผ่านตัวต้านทาน
2. เขียนเฟิร์มแวร์ให้อ่านค่าจากเซนเซอร์ แสดงผลบนจอ OLED และควบคุมไฟ LED ตามระดับแจ้งเตือน
3. ตั้งค่าให้ ESP32 เชื่อมต่อ Wi-Fi และ NETPIE ผ่าน MQTT
4. ออกแบบโครงสร้างการจัดเก็บข้อมูล ให้รองรับข้อมูลผู้เข้ารับการตรวจและค่าระดับแอลกอฮอล์ที่วัดได้
5. พัฒนาแอปพลิเคชันให้รับข้อมูลการเป่ามาแสดงผล บันทึกประวัติ และปรับค่า Threshold
6. ทดสอบความถูกต้องของการแสดงผลบนจอ OLED สถานะของไฟ LED และการรับส่งข้อมูลกับแอปพลิเคชัน
7. ออกแบบตัวเครื่องสำหรับบรรจุอุปกรณ์
8. ปรับปรุงความน่าเชื่อถือของระบบ ได้แก่ Heartbeat การตอบ ping การเก็บ Threshold ใน NVS การทำงานต่อเมื่อเน็ตหลุด และการซิงก์แบบออฟไลน์
9. ปรับปรุงความปลอดภัย ได้แก่ การเข้าสู่ระบบด้วย Firebase Authentication การกำหนด Security Rules การปกปิดเลขบัตรประชาชน และการเข้ารหัส MQTT ด้วย TLS
10. ออกแบบส่วนติดต่อผู้ใช้ใหม่ เพิ่มการตรวจ 3 ขั้น สรุปผล และการส่งออก CSV แล้วตั้งระบบตรวจโค้ดอัตโนมัติด้วย GitHub Actions

บทที่ 4
ผลการดำเนินงาน

4.1 ต้นแบบอุปกรณ์

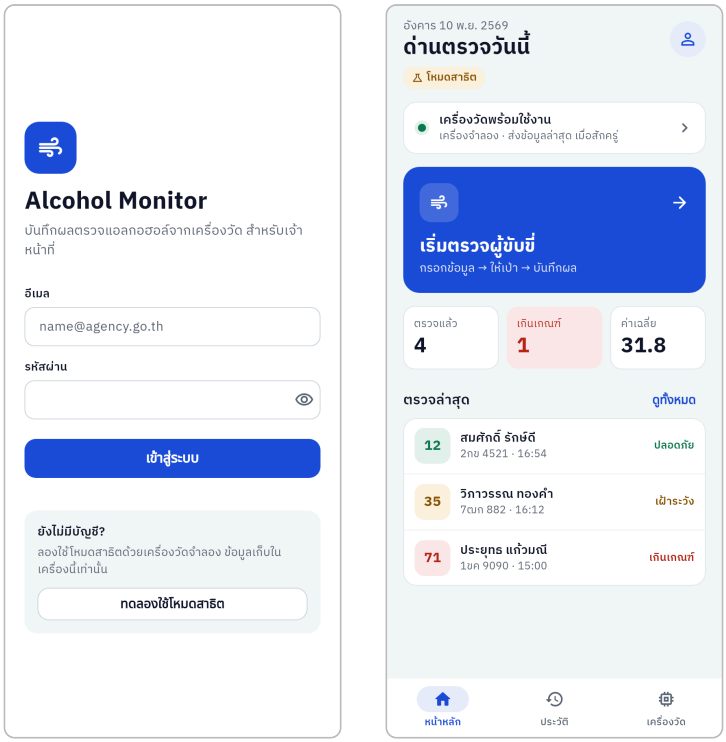
ต้นแบบอุปกรณ์ตรวจวัดแอลกอฮอล์ประกอบอยู่ในตัวเครื่อง โดยมีจอ OLED แสดงค่าที่วัดได้และไฟ LED แจ้งเตือนอยู่ด้านหน้า ดังภาพที่ 4.1



ภาพที่ 4.1 ต้นแบบอุปกรณ์ตรวจวัดแอลกอฮอล์

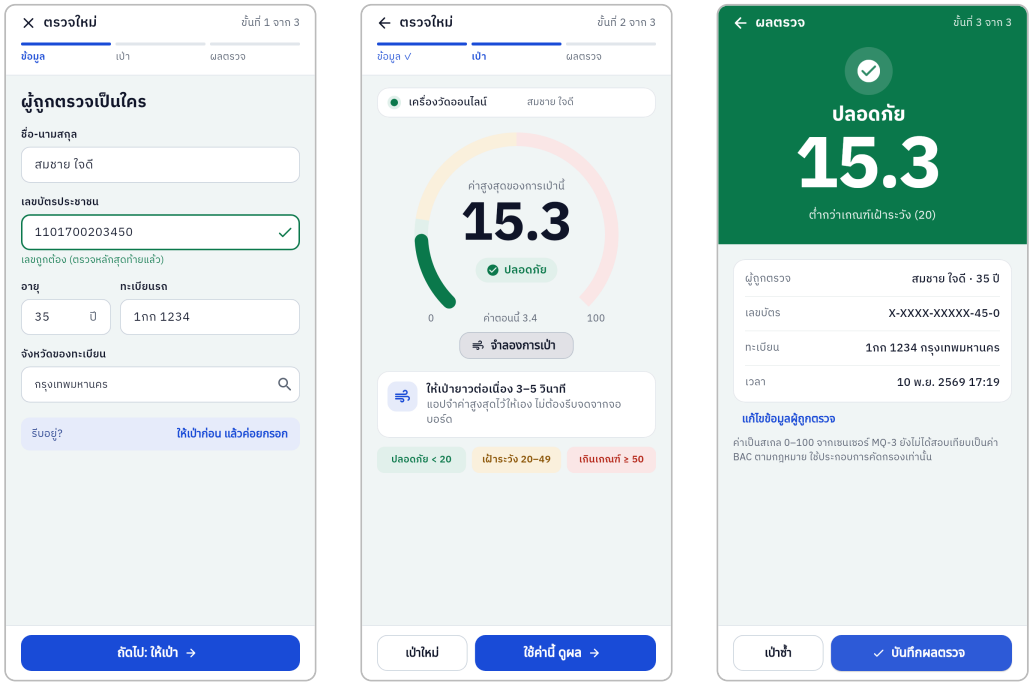
4.2 แอปพลิเคชัน

ภาพที่ 4.2 แสดงหน้าเข้าสู่ระบบและหน้าหลัก ซึ่งแสดงสถานะเครื่องวัด ปุ่มเริ่มตรวจ สรุปผลของวันนี้ และรายการตรวจล่าสุด ภาพที่ 4.3 แสดงการตรวจ 3 ขั้น ตั้งแต่กรอกข้อมูลที่ตรวจเลขบัตรประชาชนแล้ว การเป่าที่ได้ค่า 21.6 ซึ่งอยู่ในระดับเฝ้าระวัง จนถึงหน้าผลตรวจ และภาพที่ 4.4 แสดงหน้าประวัติพร้อมสรุปผล รายละเอียดรายการที่ปกปิดเลขบัตร และหน้าเครื่องวัดที่ปรับ Threshold ข้อมูลในภาพเป็นข้อมูลสมมติจากโหมดสาธิต



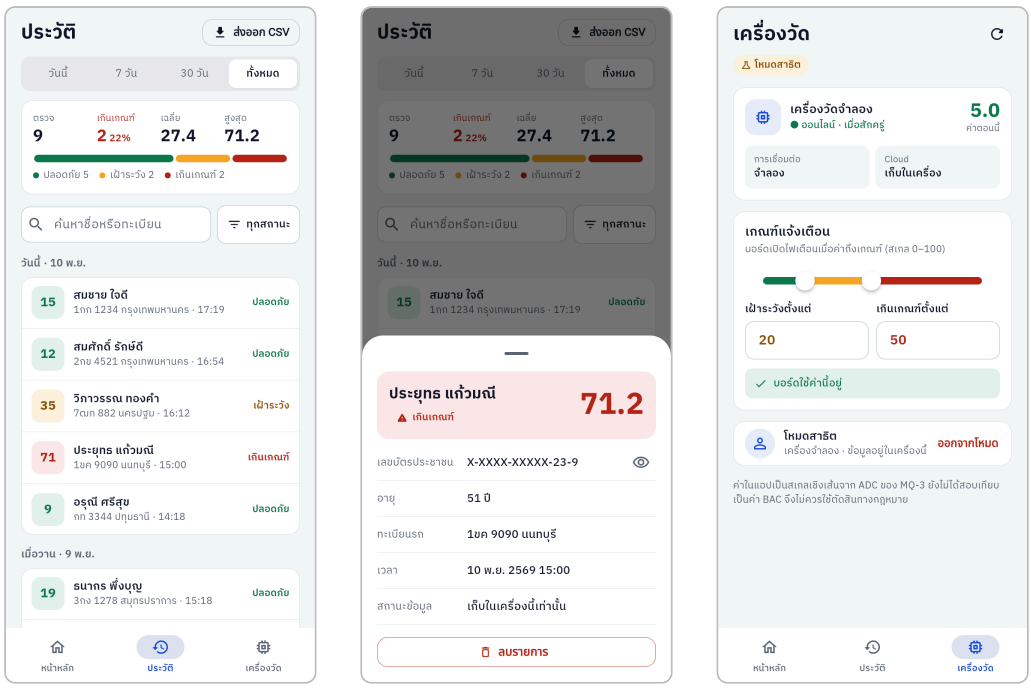
หน้าเข้าสู่ระบบ หน้าหลัก

ภาพที่ 4.2 หน้าเข้าสู่ระบบและหน้าหลักของแอปพลิเคชัน



ขั้นที่ 1 ข้อมูล ขั้นที่ 2 เป่า ขั้นที่ 3 ผลตรวจ

ภาพที่ 4.3 การตรวจผู้ขับขี่ 3 ขั้นในแอปพลิเคชัน

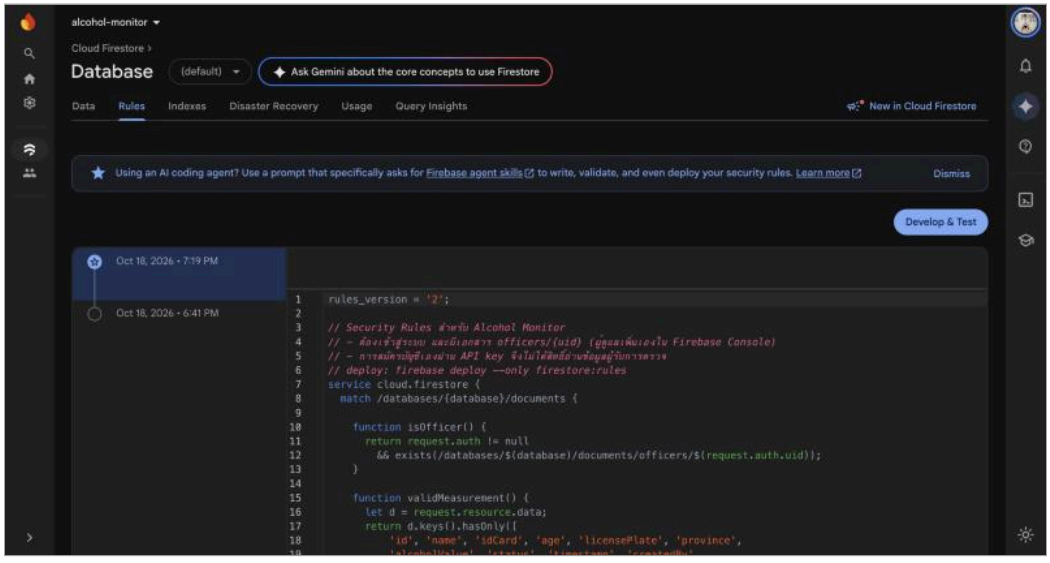


หน้าประวัติ รายละเอียดรายการ หน้าเครื่องวัด

ภาพที่ 4.4 หน้าประวัติ รายละเอียดรายการ และหน้าเครื่องวัด

4.3 การตั้งค่า Cloud Firestore

ภาพที่ 4.5 แสดง Security Rules ที่ใช้งานจริงบน Cloud Firestore ของโครงงาน ซึ่งตรงกับไฟล์ firestore.rules ในที่เก็บโค้ด โดยกำหนดให้เฉพาะเจ้าหน้าที่ที่มีเอกสารในคอลเลกชัน officers อ่านและเขียนข้อมูลได้ และตรวจรูปแบบข้อมูลตามตารางที่ 3.7



ภาพที่ 4.5 Security Rules ของ Cloud Firestore บน Firebase Console

4.4 ผลการทดสอบ

คณะผู้จัดทำทดสอบการทำงานของระบบตามฟังก์ชันหลัก ทั้งบนอุปกรณ์จริง ในโหมดสาธิต และด้วยชุดทดสอบอัตโนมัติ ผลการทดสอบแสดงดังตารางที่ 4.1

ตารางที่ 4.1 ผลการทดสอบการทำงานของระบบ

ลำดับ	รายการทดสอบ	ผลที่คาดหวัง	วิธีทดสอบและผล
1	อ่านค่าเซนเซอร์และแสดงผลบนจอ OLED และไฟ LED	จอแสดงค่าที่วัดได้ ไฟ LED ติดตามระดับ	อุปกรณ์จริง: ผ่าน (ภาพที่ 4.1)
2	รับค่าจากอุปกรณ์แบบเรียลไทม์และแสดงสถานะเครื่องวัด	แอปแสดงค่าล่าสุด และแยกสถานะออนไลน์ ไม่ส่งข้อมูลและไม่ได้เชื่อมต่อ	ทดสอบอัตโนมัติและโหมดสาธิต: ผ่าน (ภาพที่ 4.2)
3	ตรวจผู้ขับขี่ 3 ขั้นและบันทึกผล	รายการใหม่ปรากฏในหน้าประวัติ และเลขบัตรถูกปกปิด	ทดสอบอัตโนมัติและโหมดสาธิต: ผ่าน (ภาพที่ 4.3, 4.4)
4	ตรวจเลขบัตรประชาชนและข้อมูลไม่ครบ	แอปไม่ยอมบันทึก และแสดงข้อความแจ้งที่ช่องที่ผิด	ทดสอบอัตโนมัติ: ผ่าน
5	แบ่งสถานะตาม Threshold	แสดง SAFE, WARNING และ DRUNK ตามค่าที่วัดได้	ทดสอบอัตโนมัติ: ผ่าน (ภาพที่ 4.3, 4.4)
6	ปรับและส่งค่า Threshold	บอร์ดตอบกลับ และแอปแสดงว่าบอร์ดใช้ค่าใหม่แล้ว	ทดสอบอัตโนมัติ: ผ่าน (ภาพที่ 4.4)
7	บันทึกหรือลบขณะส่งขึ้นคลาวด์ไม่ได้	ข้อมูลไม่หาย ไม่กลับมาซ้ำ และซิงก์ได้ภายหลัง	ทดสอบอัตโนมัติ: ผ่าน
8	สรุปผลและส่งออก CSV	ตัวเลขสรุปถูกต้อง ไฟล์เปิดใน Excel ได้ และเลขบัตรถูกปกปิด	ทดสอบอัตโนมัติและบนเว็บ: ผ่าน
9	Security Rules ของ Firestore	ผู้ที่ไม่อยู่ใน officers อ่านไม่ได้ และข้อมูลผิดรูปแบบเขียนไม่ได้	Firestore Emulator: ผ่าน 14 กรณี
10	เชื่อมต่อ MQTT แบบ TLS และตรวจใบรับรอง	เชื่อมต่อได้เมื่อใบรับรองถูกต้อง และปฏิเสธเมื่อใบรับรองไม่ตรง	ตรวจห่วงโซ่ใบรับรองด้วย openssl: ผ่าน บนบอร์ดจริง: รอทดสอบ

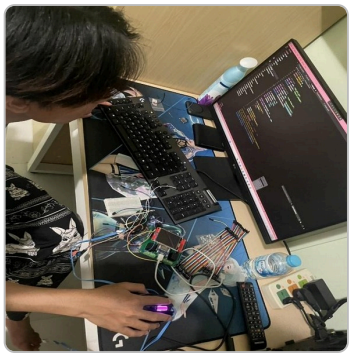
นอกจากนี้ ได้ตรวจสอบคุณภาพของซอร์สโค้ดด้วยเครื่องมือของแต่ละแพลตฟอร์ม ซึ่งรันอัตโนมัติด้วย GitHub Actions ทุกครั้งที่เปิด Pull Request ผลแสดงดังตารางที่ 4.2

ตารางที่ 4.2 ผลการตรวจสอบการคอมไพล์และคุณภาพของซอร์สโค้ด

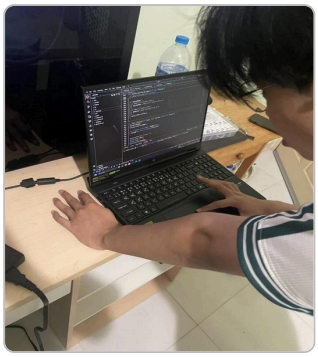
ส่วนงาน	คำสั่ง	ผลการตรวจสอบ
เฟิร์มแวร์	pio run	ผ่าน ใช้ RAM ร้อยละ 14.3 (46,920 จาก 327,680 ไบต์) และ Flash ร้อยละ 72.5 (949,893 จาก 1,310,720 ไบต์)
แอปพลิเคชัน	dart format, flutter analyze	ผ่าน ไม่พบปัญหา
แอปพลิเคชัน	flutter test	ผ่าน 53 การทดสอบ
แอปพลิเคชัน	npm test (Firestore Emulator)	ผ่าน 14 การทดสอบ
แอปพลิเคชัน	flutter build web --release	ผ่าน

ที่มา: Catboyz-45 (2026a, 2026b)

4.5 ภาพการดำเนินงาน



การเขียนโปรแกรมและต่อวงจร



การพัฒนาซอร์สโค้ด



การประกอบอุปกรณ์



การทดสอบอุปกรณ์

ภาพที่ 4.6 ภาพการดำเนินงานของคณะผู้จัดทำ

บทที่ 5
สรุปผล อภิปรายผล และข้อเสนอแนะ

5.1 สรุปผล

โครงงานระบบตรวจวัดแอลกอฮอล์ด้วยเทคโนโลยี IoT ดำเนินการได้ตามวัตถุประสงค์ ดังนี้

1. ได้ระบบที่ใช้ ESP32 ร่วมกับเซนเซอร์ MQ-3 อ่านค่าแอลกอฮอล์ในลมหายใจทุก 1 วินาที เฉลี่ยค่าจาก 10 ตัวอย่าง และแบ่งการแจ้งเตือนเป็น 3 ระดับ โดยบอร์ดยังทำงานได้แม้อินเทอร์เน็ตขาด
2. แสดงผลแบบเรียลไทม์ได้ทั้งบนจอ OLED ไฟ LED และแอปพลิเคชัน โดยแอปรับข้อมูลผ่าน MQTT แบบ TLS และรู้สถานะของเครื่องวัดจาก Heartbeat ทุก 5 วินาที
3. จัดเก็บประวัติผู้เข้ารับการตรวจบน Cloud Firestore ที่ต้องเข้าสู่ระบบและได้รับสิทธิ์ก่อน พร้อมสำเนาในเครื่องที่ซิงก์ได้เมื่อกลับมาออนไลน์
4. ได้ประยุกต์ใช้ ESP32 ร่วมกับแพลตฟอร์ม NETPIE ทั้ง MQTT และ Device Shadow รวมถึงพัฒนาแอปพลิเคชันหลายแพลตฟอร์มด้วย Flutter
5. ได้เครื่องมือคัดกรองเบื้องต้นที่ปรับเกณฑ์ได้ตามกลุ่มผู้ขับขี่ ตรวจเป็นขั้นตอนที่ลดความผิดพลาด สรุปผลและส่งออกข้อมูลได้ เพื่อต่อยอดเป็นอุปกรณ์พกพาหรือใช้ที่จุดคัดกรอง

5.2 อภิปรายผล

การออกแบบให้ปรับ Threshold ได้จากแอปพลิเคชันช่วยให้ระบบรองรับเกณฑ์ตามกฎหมายที่ต่างกันระหว่างกลุ่มผู้ขับขี่ในตารางที่ 2.1 อย่างไรก็ตาม ค่าที่ระบบแสดงเป็นการแปลงค่าดิบจาก ADC แบบเชิงเส้น ขณะที่ความต้านทานของเซนเซอร์ MQ-3 เปลี่ยนแปลงไม่เป็นเชิงเส้นตามความเข้มข้นของแอลกอฮอล์ (Hanwei Electronics, n.d.) ค่าดัชนีจึงใช้เปรียบเทียบระดับมากหรือน้อยได้ แต่ยังไม่สามารถเทียบเป็นมิลลิกรัมเปอร์เซ็นต์ได้โดยตรง

ในด้านการสื่อสาร การเปลี่ยนจากการอ่าน Device Shadow เป็นระยะมาใช้ MQTT ร่วมกับ Heartbeat และ ping ทำให้แอปได้ข้อมูลทันทีและแยกได้ว่าบอร์ดออนไลน์หรือเงียบไป แทนที่จะดูเพียงว่า API ตอบกลับหรือไม่ ส่วนการออกแบบแบบ Offline-first ช่วยให้บันทึกข้อมูลได้แม้สัญญาณอินเทอร์เน็ตที่จุดตรวจไม่เสถียร

ในด้านการใช้งาน การแบ่งการตรวจเป็น 3 ขั้นและล็อกค่าที่ใช้ทำให้ผลที่บันทึกตรงกับการเป่าของผู้ถูกตรวจคนนั้น และการแสดงผลด้วยสีสถานะเต็มครึ่งจอช่วยให้อ่านผลได้จากระยะไกล ในด้านความปลอดภัย การเข้ารหัส MQTT การจำกัดสิทธิ์ด้วย Security Rules และการปกปิดเลขบัตรประชาชนช่วยลดความเสี่ยงต่อข้อมูลส่วนบุคคลตามแนวทางของพระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562

5.3 ปัญหาและข้อจำกัด

1. ระบบยังไม่ได้สอบเทียบกับเครื่องวัดแอลกอฮอล์มาตรฐาน จึงใช้ได้เพื่อการคัดกรองเบื้องต้นเท่านั้น ไม่สามารถใช้เป็นหลักฐานทางกฎหมาย
2. ค่าที่วัดได้จากเซนเซอร์ MQ-3 ขึ้นกับอุณหภูมิ ความชื้น และระยะเวลาการอุ่นเซนเซอร์

3. รหัสอุปกรณ์ NETPIE ฝังแอปที่ส่งผ่าน --dart-define จะติดอยู่ในแอปที่ build ออกไป หากถูกแกะออกไป ผู้อื่นจะอ่านค่าหรือส่ง Threshold ได้ แต่ปลอมเป็นบอร์ดไม่ได้
4. ประวัติในเครื่องเก็บใน SharedPreferences แบบไม่เข้ารหัส และรายการที่ยังรอซิงก์ยังมีเลขบัตรประชาชนเต็ม
5. ใบรับรองหลักฝังอยู่ในเฟิร์มแวร์ หากผู้ให้บริการเปลี่ยนผู้ออกใบรับรอง ต้องอัปเดตไฟล์ ca_certs.h และแฟลชบอร์ดใหม่ และเฟิร์มแวร์เวอร์ชันที่ใช้ TLS ยังต้องทดสอบบนบอร์ดจริง
6. ระบบยังไม่มีขั้นตอนขอความยินยอมจากผู้ถูกตรวจ ซึ่งจำเป็นหากนำไปใช้งานจริง

5.4 ข้อเสนอแนะ

1. สอบเทียบเซนเซอร์กับเครื่องวัดมาตรฐาน และสร้างสูตรแปลงค่าจากข้อมูลการทดลองจริง เพื่อแสดงผลเป็นหน่วยที่ตรวจสอบได้
2. เพิ่มเซิร์ฟเวอร์กลาง (Backend) ที่ถือรหัสอุปกรณ์ NETPIE แทนแอปพลิเคชัน และตรวจสิทธิ์ผู้ใช้ก่อนส่งคำสั่งไปยังบอร์ด
3. เพิ่มการกำหนดสิทธิ์ตามบทบาท เช่น เจ้าหน้าที่และผู้ดูแล และบันทึกประวัติการเข้าถึงข้อมูล
4. เข้ารหัสข้อมูลที่เก็บในเครื่อง และเพิ่มขั้นตอนขอความยินยอมจากผู้ถูกตรวจ
5. เพิ่มวงจรแบ่งแรงดันระหว่างเซนเซอร์กับ ESP32 การจัดเก็บผลวัดชั่วคราวในบอร์ดเมื่ออินเทอร์เน็ตขาด และการอัปเดตเฟิร์มแวร์ผ่านเครือข่าย (OTA) ที่ตรวจลายเซ็น

บรรณานุกรม

กฎกระทรวง ฉบับที่ 21 (พ.ศ. 2560) ออกตามความในพระราชบัญญัติจราจรทางบก พ.ศ. 2522. (2560). *ราชกิจจานุเบกษา*.

พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562. (2562, 27 พฤษภาคม). *ราชกิจจานุเบกษา*. เล่ม 136 ตอนที่ 69 ก.

มติชนออนไลน์. (2560, 31 พฤษภาคม). *ประกาศใช้ กม.เมาแล้วขับ เด็กต่ำกว่า 20 ปี–ใบขับขี่ชั่วคราว แอลกอฮอล์เกิน 20 มก. ถูกจับ*. สืบค้นจาก https://www.matichon.co.th/local/news_571560

Banks, A., & Gupta, R. (Eds.). (2014). *MQTT version 3.1.1* (OASIS Standard). OASIS. Retrieved from http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

Blanchon, B. (n.d.). *ArduinoJson*. Retrieved from https://arduinojson.org

Catboyz-45. (2026a). *alcohol-monitor-app* [Source code]. GitHub. Retrieved from https://github.com/Catboyz-45/alcohol-monitor-app

Catboyz-45. (2026b). *alcohol-monitor-firmware* [Source code]. GitHub. Retrieved from https://github.com/Catboyz-45/alcohol-monitor-firmware

Espressif Systems. (n.d.-a). *ESP32 series datasheet*. Retrieved from https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

Espressif Systems. (n.d.-b). Analog to digital converter (ADC). In *ESP-IDF programming guide*. Retrieved from https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/adc_oneshot.html

Espressif Systems. (n.d.-c). Non-volatile storage library. In *ESP-IDF programming guide*. Retrieved from https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/nvs_flash.html

Google. (n.d.-a). Use the Cloud Firestore REST API. In *Firebase documentation*. Retrieved from https://firebase.google.com/docs/firestore/use-rest-api

Google. (n.d.-b). Firebase Realtime Database REST API. In *Firebase documentation*. Retrieved from https://firebase.google.com/docs/database/rest/start

Google. (n.d.-c). *Flutter documentation*. Retrieved from https://docs.flutter.dev

Google. (n.d.-d). Firebase Auth REST API. In *Firebase documentation*. Retrieved from https://firebase.google.com/docs/reference/rest/auth

Google. (n.d.-e). Get started with Cloud Firestore Security Rules. In *Firebase documentation*. Retrieved from https://firebase.google.com/docs/firestore/security/get-started

Hanwei Electronics. (n.d.). *Technical data MQ-3 gas sensor*. Retrieved from https://www.sparkfun.com/datasheets/Sensors/MQ-3.pdf

International Telecommunication Union. (2012). *Overview of the Internet of things* (ITU-T Recommendation Y.2060). Retrieved from https://www.itu.int/rec/T-REC-Y.2060-

201206-I

NETPIE. (n.d.). *NETPIE documentation*. Retrieved from https://docs.netpie.io

O'Leary, N. (n.d.). *Arduino Client for MQTT (PubSubClient)*. Retrieved from https://pubsubclient.knolleary.net

PlatformIO Labs. (n.d.). *PlatformIO documentation*. Retrieved from https://docs.platformio.org

Rescorla, E. (2018). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). Internet Engineering Task Force. Retrieved from https://www.rfc-editor.org/rfc/rfc8446

Rousselet, R. (n.d.). *provider* [Dart package]. pub.dev. Retrieved from https://pub.dev/packages/provider

Solomon Systech. (2008). *SSD1306: 128 x 64 dot matrix OLED/PLED segment/common driver with controller* (Rev. 1.1). Retrieved from https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf

ภาคผนวก ก
ที่เก็บซอร์สโค้ดและวิธีติดตั้ง

ซอร์สโค้ดของโครงงานเผยแพร่บน GitHub แยกเป็น 2 ที่เก็บ และมีเว็บเดโมให้ทดลองใช้ ได้แก่

1. เฟิร์มแวร์ ESP32: https://github.com/Catboyz-45/alcohol-monitor-firmware
2. แอปพลิเคชัน Flutter: https://github.com/Catboyz-45/alcohol-monitor-app
3. เว็บเดโม: https://catboyz-45.github.io/alcohol-monitor-app/

ก.1 การติดตั้งเฟิร์มแวร์

```
git clone https://github.com/Catboyz-45/alcohol-monitor-firmware.git
cd alcohol-monitor-firmware
cp include/secrets.example.h include/secrets.h
# แก้ไข secrets.h ให้เป็นค่า Wi-Fi, NETPIE และ Firebase ของตนเอง
pio run --target upload
pio device monitor --baud 115200
```

ก.2 การติดตั้งแอปพลิเคชัน

```
git clone https://github.com/Catboyz-45/alcohol-monitor-app.git
cd alcohol-monitor-app
flutter pub get
flutter run -d chrome          # ไม่ตั้งค่าอะไร จะเปิดเป็นโหมดสาธิต

cp config.example.json config.json
# แก้ไข config.json ให้เป็นค่า NETPIE (อุปกรณ์ฝั่งแอป) และ Firebase ของตนเอง
flutter run --dart-define-from-file=config.json
```

ก.3 การตั้งค่า Firebase

1. เปิดการเข้าสู่ระบบแบบ Email/Password ใน Firebase Authentication และสร้างบัญชีเจ้าหน้าที่
2. สร้างเอกสาร officers/{uid} ใน Cloud Firestore ให้แต่ละบัญชีที่อนุญาต
3. Deploy Security Rules ด้วยคำสั่ง `firebase deploy --only firestore:rules`

ภาคผนวก ข

ซอร์สโค้ดเฟิร์มแวร์ (src/main.cpp)

```
#include <Arduino.h>
#include <ArduinoJson.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <HTTPClient.h>
#include <Preferences.h>
#include <PubSubClient.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <Wire.h>

#include "ca_certs.h"
#include "secrets.h"

// ส่งผลวัดเข้า Firebase Realtime Database (แอปไม่ได้อ่านส่วนนี้ ใช้ Firestore แทน)
// ปิดไว้เป็นค่าเริ่มต้น เพราะต้องเปิด RTDB rules ให้เขียนได้โดยไม่ล็อกอิน
// และ HTTPS แบบ blocking ทำให้ loop ค้าง เปิดได้ด้วย build flag -D ENABLE_FIREBASE_RTDB=1
#ifndef ENABLE_FIREBASE_RTDB
#define ENABLE_FIREBASE_RTDB 0
#endif

const char *mqttServer = "mqtt.netpie.io";
// MQTT ผ่าน TLS และตรวจ certificate ของ NETPIE (เดิมใช้ 1883 ซึ่งส่ง token และข้อมูลแบบไม่เข้ารหัส)
const int mqttPort = 8883;

const char *topic_sub = "@msg/home/device_control";
const char *topic_pub = "@msg/home/device_status";
const char *data_pub = "@shadow/data/update";

const int OLED_SDA = 21;
const int OLED_SCL = 22;
const int LED1_PIN = 4;
const int LED2_PIN = 19;
const int MQ3_PIN = 34;

const uint8_t SCREEN_WIDTH = 128;
const uint8_t SCREEN_HEIGHT = 64;
const int OLED_RESET = -1;
const uint8_t OLED_ADDR = 0x3C;

const unsigned long SENSOR_READ_INTERVAL_MS = 1000; // อ่านเซ็นเซอร์ทุก 1 วินาที
const unsigned long DISPLAY_HOLD_MS = 2500; // แสดงค่าที่วัดได้ 2.5 วินาที ก่อนกลับไปหน้ารอวัดใหม่
const unsigned long HEARTBEAT_INTERVAL_MS = 5000; // ส่งสถานะให้แอปอย่างน้อยทุก 5 วินาที แม้ไม่มีกา
รเป่า
const unsigned long MQTT_RETRY_INTERVAL_MS = 5000; // ลองต่อ MQTT ใหม่ทุก 5 วินาทีโดยไม่บล็อก
loop
const unsigned long WIFI_CONNECT_TIMEOUT_MS = 15000;
const unsigned long TLS_HANDSHAKE_TIMEOUT_S = 10; // กัน loop ค้างนานตอนเน็ตช้า
const float MIN_REPORT_BAC = 10.0f;
const float BAC_MAX_VALUE = 100.0f;

float threshold1 = 20.0f;
float threshold2 = 50.0f;
```

```
float lastBacValue = 0.0f;
float lastDisplayedBac = 0.0f;
int lastRawValue = 0;
int alertLevel = 0;
unsigned long lastSensorReadAt = 0;
unsigned long lastDisplayValueAt = 0;
unsigned long lastStatusPublishAt = 0;
unsigned long lastMqttAttemptAt = 0;
bool displayReady = false;
bool displayHoldingValue = false;

WiFiClientSecure mqttSecureClient;
PubSubClient mqttClient(mqttSecureClient);
Preferences preferences;
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// จำกัดค่า BAC ให้อยู่ในช่วงที่ระบบรองรับ
float clampBac(float value) {
  if (value < 0.0f) {
    return 0.0f;
  }
  if (value > BAC_MAX_VALUE) {
    return BAC_MAX_VALUE;
  }
  return value;
}

// ปัดทศนิยมให้เหลือ 1 ตำแหน่งเพื่อแสดงผล/ส่งข้อมูลให้สม่ำเสมอ
float roundOneDecimal(float value) {
  return roundf(value * 10.0f) / 10.0f;
}

// จัดระเบียบค่า threshold ให้อยู่ในช่วงถูกต้องและเรียงจากน้อยไปมาก
void normalizeThresholds() {
  threshold1 = clampBac(threshold1);
  threshold2 = clampBac(threshold2);

  // ถ้าสลับลำดับกันอยู่ ให้สลับกลับเพื่อกันเงื่อนไขผิดพลาด
  if (threshold1 > threshold2) {
    const float temp = threshold1;
    threshold1 = threshold2;
    threshold2 = temp;
  }
}

// โหลด threshold ที่แอปเคยตั้งไว้ ไม่ให้รีเซ็ตเป็น 20/50 ทุกครั้งที่บอร์ดรีบูต
void loadThresholds() {
  preferences.begin("alcohol", true);
  threshold1 = preferences.getFloat("t1", threshold1);
  threshold2 = preferences.getFloat("t2", threshold2);
  preferences.end();
  normalizeThresholds();
}

void saveThresholds() {
  preferences.begin("alcohol", false);
  preferences.putFloat("t1", threshold1);
  preferences.putFloat("t2", threshold2);
  preferences.end();
```

```
}

// อ่านค่าเซ็นเซอร์หลายครั้งแล้วเฉลี่ย เพื่อลดสัญญาณรบกวน
int readAlcoholRaw() {
  long total = 0;
  const int sampleCount = 10;

  for (int sample = 0; sample < sampleCount; sample++) {
    total += analogRead(MQ3_PIN);
    delay(5);
  }

  return total / sampleCount;
}

// แปลงค่าดิบเป็นเปอร์เซ็นต์ ตามสเกล 0–100
float convertRawToBac(int rawValue) {
  return clampBac((rawValue / 4095.0f) * BAC_MAX_VALUE);
}

// วาดข้อมูลสถานะทั้งหมดลง OLED
void renderDisplay(float bacValue, bool showMeasuredValue) {
  // ถ้า OLED ไม่พร้อม ให้ข้ามทันที
  if (!displayReady) {
    return;
  }

  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Alcohol Monitor");

  display.setTextSize(2);
  display.setCursor(0, 16);
  display.print("Alc: ");
  display.println(roundOneDecimal(showMeasuredValue ? bacValue : 0.0f), 1);

  display.setTextSize(1);
  display.setCursor(0, 42);
  display.print("T1:");
  display.print(roundOneDecimal(threshold1), 0);
  display.print(" T2:");
  display.println(roundOneDecimal(threshold2), 0);

  display.setCursor(0, 54);
  if (showMeasuredValue) {
    display.print("Sent to cloud");
  } else {
    display.print("Waiting for sample");
  }

  display.display();
}

// อัปเดตระดับแจ้งเตือน และควบคุม LED ตามค่า BAC
void updateAlertOutputs(float bacValue) {
  if (bacValue >= threshold2) {
    alertLevel = 2;
```

```
  } else if (bacValue >= threshold1) {
    alertLevel = 1;
  } else {
    alertLevel = 0;
  }

  digitalWrite(LED1_PIN, alertLevel >= 1 ? HIGH : LOW);
  digitalWrite(LED2_PIN, alertLevel >= 2 ? HIGH : LOW);
}

// ส่งสถานะล่าสุดขึ้น Shadow ของ NetPIE
void publishShadow() {
  if (!mqttClient.connected()) {
    Serial.println("Shadow publish skipped: MQTT disconnected");
    return;
  }

  const int led1State = digitalRead(LED1_PIN);
  const int led2State = digitalRead(LED2_PIN);

  StaticJsonDocument<384> doc;
  JsonObject data = doc.createNestedObject("data");
  data["red"] = led2State;
  data["yellow"] = led1State;
  data["bac"] = roundOneDecimal(lastBacValue);
  data["raw"] = lastRawValue;
  data["alertLevel"] = alertLevel;
  data["threshold1"] = roundOneDecimal(threshold1);
  data["threshold2"] = roundOneDecimal(threshold2);

  char payload[384];
  serializeJson(doc, payload, sizeof(payload));
  const bool ok = mqttClient.publish(data_pub, payload, true);
  Serial.print("Shadow publish: ");
  Serial.println(ok ? "OK" : "FAIL");
}

// ส่งผลวัดและสถานะไปที่ @msg/home/device_status ให้แอป
// ใช้ทั้งตอนมีการเป่า, heartbeat ทุก HEARTBEAT_INTERVAL_MS และตอบ ping จากแอป
void publishStatus() {
  if (!mqttClient.connected()) {
    Serial.println("MQTT publish skipped: MQTT disconnected");
    return;
  }
  lastStatusPublishAt = millis();

  StaticJsonDocument<256> doc;
  doc["bac"] = roundOneDecimal(lastBacValue);
  doc["raw"] = lastRawValue;
  doc["threshold1"] = roundOneDecimal(threshold1);
  doc["threshold2"] = roundOneDecimal(threshold2);
  doc["alertLevel"] = alertLevel;
  doc["millis"] = millis();

  char payload[256];
  serializeJson(doc, payload, sizeof(payload));
  if (!mqttClient.publish(topic_pub, payload)) {
    Serial.println("MQTT status publish: FAIL");
  }
```

```
}

// ส่งค่าการวัดไปเก็บใน Firebase Realtime Database
void sendToFirebase() {
#if ENABLE_FIREBASE_RTDB
  // ส่งเฉพาะตอนต่อ WiFi ได้เท่านั้น
  if (WiFi.status() != WL_CONNECTED) {
    return;
  }

  WiFiClientSecure secureClient;
  secureClient.setCACert(GOOGLE_ROOT_CA);
  secureClient.setHandshakeTimeout(TLS_HANDSHAKE_TIMEOUT_S);

  HTTPClient https;
  const String url = String(Secrets::FIREBASE_BASE_URL) + "/measurements.json";
  if (!https.begin(secureClient, url)) {
    Serial.println("Firebase connection failed");
    return;
  }

  https.setTimeout(3000);
  https.addHeader("Content-Type", "application/json");

  StaticJsonDocument<256> doc;
  doc["bac"] = roundOneDecimal(lastBacValue);
  doc["raw"] = lastRawValue;
  doc["threshold1"] = roundOneDecimal(threshold1);
  doc["threshold2"] = roundOneDecimal(threshold2);
  doc["alertLevel"] = alertLevel;
  doc["millis"] = millis();

  String body;
  serializeJson(doc, body);
  const int httpCode = https.POST(body);
  Serial.print("Firebase HTTP code: ");
  Serial.println(httpCode);
  https.end();
#endif
}

// รับค่า threshold ใหม่จาก JSON และบังคับให้ค่าถูกต้องก่อนใช้งาน
void applyThresholdUpdate(JsonObjectConst source) {
  bool updated = false;

  // รับเฉพาะค่าที่เป็นตัวเลข กันค่าแปลก ๆ กลายเป็น 0 แล้วไฟเตือนติดตลอด
  if (source["threshold1"].is<float>()) {
    threshold1 = source["threshold1"].as<float>();
    updated = true;
  }
  if (source["threshold2"].is<float>()) {
    threshold2 = source["threshold2"].as<float>();
    updated = true;
  }

  if (!updated) {
    return;
  }
```

```
  normalizeThresholds();
  saveThresholds();
  updateAlertOutputs(lastBacValue);
  publishShadow();
  // ส่งสถานะทันที เพื่อให้แอปยืนยันได้ว่าบอร์ดรับค่าใหม่แล้ว
  publishStatus();
  renderDisplay(lastDisplayedBac, displayHoldingValue);

  Serial.print("Threshold updated: ");
  Serial.print(threshold1);
  Serial.print(", ");
  Serial.println(threshold2);
}

// callback เมื่อมีข้อความ MQTT เข้ามา
void callback(char *topic, byte *payload, unsigned int length) {
  (void)topic;

  String message;
  for (unsigned int index = 0; index < length; index++) {
    message += static_cast<char>(payload[index]);
  }

  Serial.print("Message arrived: ");
  Serial.println(message);

  StaticJsonDocument<256> doc;
  DeserializationError error = deserializeJson(doc, message);
  if (error) {
    Serial.print("Invalid JSON: ");
    Serial.println(error.c_str());
    return;
  }

  JsonObjectConst source = doc.as<JsonObjectConst>();
  if (doc["data"].is<JsonObjectConst>()) {
    source = doc["data"].as<JsonObjectConst>();
  }

  // แอปส่ง {"ping": 1} ตอนเชื่อมต่อ เพื่อขอสถานะทันทีโดยไม่ต้องรอ heartbeat
  if (source.containsKey("ping")) {
    publishStatus();
  }

  applyThresholdUpdate(source);
}

// เชื่อมต่อ WiFi โดยรอไม่เกิน WIFI_CONNECT_TIMEOUT_MS
// ถ้ายังต่อไม่ได้ก็ทำงานต่อ (อ่านเซนเซอร์/ไฟเตือนยังใช้ได้) และให้ ESP32 ต่อใหม่เอง
void setupWifi() {
  delay(10);
  Serial.print("Connecting to ");
  Serial.println(Secrets::WIFI_SSID);
  WiFi.mode(WIFI_STA);
  WiFi.setAutoReconnect(true);
  WiFi.begin(Secrets::WIFI_SSID, Secrets::WIFI_PASSWORD);

  const unsigned long startedAt = millis();
  while (WiFi.status() != WL_CONNECTED &&
```

```
         millis() - startedAt < WIFI_CONNECT_TIMEOUT_MS) {
    delay(500);
    Serial.print(".");
  }

  Serial.println(WiFi.status() == WL_CONNECTED
                     ? "\nWiFi connected"
                     : "\nWiFi not connected yet, continuing offline");
}

// ลองเชื่อม MQTT ใหม่ทีละครั้งตามรอบเวลา ไม่วนรอจนบอร์ดค้าง
// (เดิมวน while + delay ทำให้ไฟเตือนบนบอร์ดหยุดทำงานตอนเน็ตหลุด)
void reconnectMQTT() {
  if (WiFi.status() != WL_CONNECTED) {
    return;
  }
  const unsigned long now = millis();
  if (lastMqttAttemptAt != 0 && now - lastMqttAttemptAt < MQTT_RETRY_INTERVAL_MS) {
    return;
  }
  lastMqttAttemptAt = now;

  Serial.println("Attempting MQTT connection...");
  if (mqttClient.connect(
          Secrets::MQTT_CLIENT_ID,
          Secrets::MQTT_USERNAME,
          Secrets::MQTT_PASSWORD)) {
    Serial.println("Connected to NetPIE");
    mqttClient.subscribe(topic_sub);
    publishShadow();
    publishStatus();
  } else {
    Serial.print("MQTT failed, rc=");
    Serial.println(mqttClient.state());
    char tlsError[100];
    if (mqttSecureClient.lastError(tlsError, sizeof(tlsError)) != 0) {
      Serial.print("TLS error: ");
      Serial.println(tlsError);
    }
  }
}

// เริ่มต้นจอ OLED และแสดงหน้าเริ่มต้น
void setupDisplay() {
  Wire.begin(OLED_SDA, OLED_SCL);
  displayReady = display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR);

  if (!displayReady) {
    Serial.println("OLED not found");
    return;
  }

  renderDisplay(0.0f, false);
}

// ตั้งค่าฮาร์ดแวร์และบริการทั้งหมดตอนบูต
void setup() {
  Serial.begin(115200);
```

```
  pinMode(LED1_PIN, OUTPUT);
  pinMode(LED2_PIN, OUTPUT);
  pinMode(MQ3_PIN, INPUT);

  digitalWrite(LED1_PIN, LOW);
  digitalWrite(LED2_PIN, LOW);

  analogReadResolution(12);
  analogSetPinAttenuation(MQ3_PIN, ADC_11db);
  loadThresholds();

  setupDisplay();
  setupWifi();
  // ตั้งเวลาจาก NTP ไว้ให้ตรวจวันหมดอายุ certificate ได้ (ทำงานเบื้องหลัง ไม่บล็อก)
  configTime(0, 0, "pool.ntp.org", "time.google.com");

  mqttSecureClient.setCACert(NETPIE_ROOT_CA);
  mqttSecureClient.setHandshakeTimeout(TLS_HANDSHAKE_TIMEOUT_S);
  mqttClient.setServer(mqttServer, mqttPort);
  mqttClient.setCallback(callback);
  mqttClient.setBufferSize(512);
}

// วนลูปหลัก: ดูแลการเชื่อมต่อ, อ่านเซ็นเซอร์, อัปเดตแจ้งเตือน, และส่งข้อมูล
void loop() {
  // ถ้าหลุด MQTT ให้ลองเชื่อมใหม่ แต่ยังอ่านเซนเซอร์และคุมไฟเตือนต่อไป
  if (!mqttClient.connected()) {
    reconnectMQTT();
  }
  mqttClient.loop();

  const unsigned long now = millis();
  // อ่านค่าเซ็นเซอร์ตามช่วงเวลา ไม่อ่านถี่เกินจำเป็น
  if (now - lastSensorReadAt >= SENSOR_READ_INTERVAL_MS) {
    lastSensorReadAt = now;

    lastRawValue = readAlcoholRaw();
    lastBacValue = roundOneDecimal(convertRawToBac(lastRawValue));
    updateAlertOutputs(lastBacValue);
    publishShadow();

    Serial.print("BAC: ");
    Serial.println(lastBacValue);

    // ส่งขึ้น cloud เฉพาะค่าที่เกินเกณฑ์ขั้นต่ำ
    if (lastBacValue > MIN_REPORT_BAC) {
      lastDisplayedBac = lastBacValue;
      lastDisplayValueAt = now;
      displayHoldingValue = true;
      renderDisplay(lastDisplayedBac, true);
      publishStatus();
      sendToFirebase();
    } else if (now - lastStatusPublishAt >= HEARTBEAT_INTERVAL_MS) {
      publishStatus();
    }
  }

  // ครบเวลาค้างหน้าจอแล้วกลับไปสถานะรอวัด
  if (displayHoldingValue && (now - lastDisplayValueAt >= DISPLAY_HOLD_MS)) {
```

```
    displayHoldingValue = false;
    lastDisplayedBac = 0.0f;
    renderDisplay(0.0f, false);
  }
}
```